UNIT-1

Introduction: Basic Terminology: Elementary Data Organization

Data is defined as a collection of individual facts or statistics. Data can come in the form of text, observations, figures, images, numbers, graphs, or symbols.

Data is a raw form of knowledge and, on its own, doesn't carry any significance or purpose. Data can be simple—and may even seem useless until it is analyzed, organized, and interpreted.

There are two main types of data:

- **Quantitative data** is provided in numerical form, like the weight, volume, or cost of an item.
- **Qualitative data** is descriptive, but non-numerical, like the name, sex, or eye .

Information

information is the result of analyzing and interpreting pieces of data. Whereas data is the individual figures, numbers, or graphs, information is the perception of those pieces of knowledge. when the data is organized and compiled in a useful way can it provide information that is beneficial to others.

Entity: An entity is something that has certain attributes or properties which may be assigned values. The values may be either numeric or non-numeric.

Ex: Attributes-	Names,	Age,	Sex,
	Rohland Gail,	34,	F,

Field is a single elementary unit of information representing an attribute of an entity. Record is the collection of field values of a given entity.

File is the collection of records of the entities in a given entity set **Records** may also be classified according to length.

A file can have fixed-length records or variable-length records.

- In fixed-length records, all the records contain the same data items with the same amount of space assigned to each data item.
- In variable-length records file records may contain different lengths.

Data type

A **data type** is a classification of data which tells the compiler or interpreter how the programmer intends to use the data. Most programming languages support various types of data, including integer, float, character or string, and Boolean.

Or

1. Data type defines a certain domain of values
2. Defines operations allowed on those values. Example: int type
   Takes only integer values
   Operations add sub mul bitwise operations etc

Abstract Data Type (ADT)- ADT is a mathematical model that logically represents a datatype. ADT only tells us the essential features without including background details. It specifies set of data and set of operation that can be performed on the data. Example: List adt, Stack adt. Queue adt..

Data structure

A data structure is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. It refers to the logical or mathematical representation of data, as well as the implementation in a computer program. Data structures are normally divided into two broad categories

1. primitive Data Structures(built-in)
2. Non-Primitive Data Structures(user defined)

Primitive data structure:

primitive data structure is considered as the fundamental data structure and it allows storing the values of only one data type. The primitive data structure consists of fundamental data types like float, character, integer, etc.

Non-Primitive Data Structures:

non-primitive data structures are considered as the user-defined structure that allows storing values of different data types within one entity.
The non-primitive data structure is further classified into two parts i.e. linear data structure and non-linear data structure. Linear data structures are considered sequential data structures and sequential data structures store elements in a sequence in the memory.
Linked List, Array, and Stack are some examples of linear data structure which stores the elements sequentially. The non-linear data structure is considered as a random type of data structure. Graphs and trees are examples of non-linear data structures.

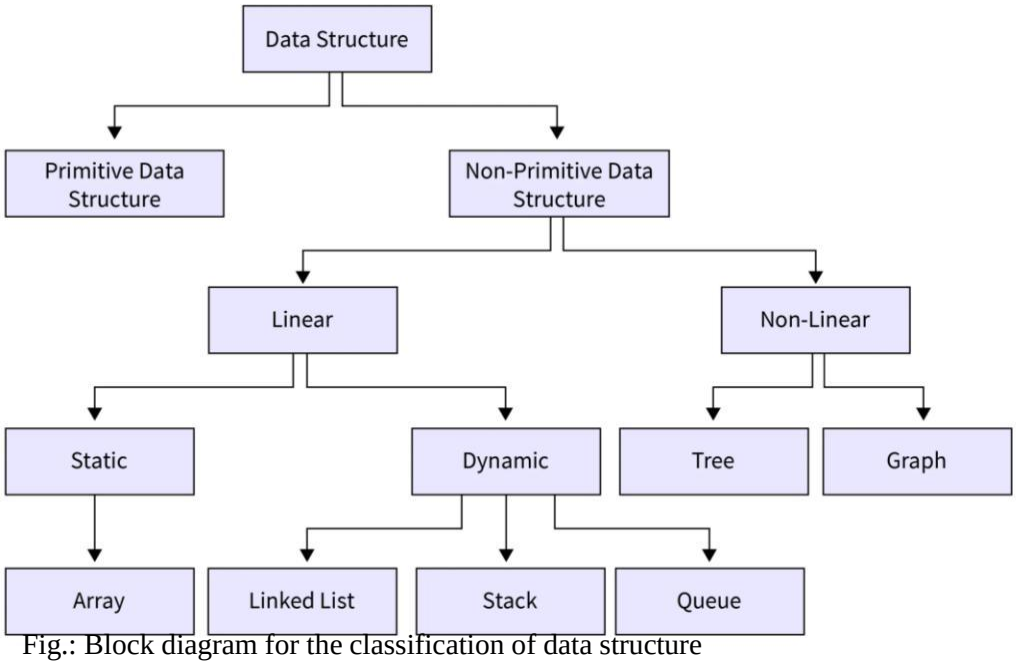


Fig.: Block diagram for the classification of data structure

Difference Between Primitive and Non-Primitive Data Structure

The difference between primitive and nonprimitive data structures is explained below:

Primitive data structure	Non Primitive Data structure
Primitive data structure is the data structure that allows you to store only single data type values.	Non-Primitive data structure is a data structure that allows you to store multiple data type values.

Primitive data structure	Non Primitive Data structure
integer, boolean, character, float, etc. are some examples of primitive data structures.	Array, Linked List, Stack, etc. are some examples of non-primitive data structures.
Primitive data structure always contains some value i.e. these data structures do not allow you to store NULL values.	You can store a NULL value in the non-primitive data structures.
The size of the primitive data structures is dependent on the type of the primitive data structure.	The size of the non-primitive data structure is not fixed.

Algorithm: An algorithm is well-defined steps by step procedure that take some value or set of values as input and produce some value or set of values as output.
it must have the following characteristics:

- **Clear and Unambiguous**: The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-defined inputs**: If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Finite-ness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.
- **Input**: An algorithm has zero or more inputs. Each that contains a fundamental operator must accept zero or more inputs.
- **Output**: An algorithm produces at least one output. Every instruction that contains a fundamental operator must accept zero or more inputs.
- **Definiteness:** All instructions in an algorithm must be unambiguous, precise, and easy to interpret. By referring to any of the instructions in an algorithm one can clearly understand what is to be done. Every fundamental operator in instruction must be defined without any ambiguity.
- **Finiteness:** An algorithm must terminate after a finite number of steps in all test cases. Every instruction which contains a fundamental operator must be terminated within a finite amount of time. Infinite loops or recursive functions without base conditions do not possess finiteness.

- **Effectiveness:** An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil.

Complexity:

Complexity of an algorithm Is a function of size of input of a given problem. Instance, which determines how much running time oblique space is needed by the algorithm in order to run to compilation.
Time complexity: time complexity of an algorithm is the amount of time it needs in order to run to compilation.
Space complexity: space complexity of an algorithm is the amount of space it needs in order to run to compilation.

Algorithm Efficiency

The efficiency of an algorithm is mainly defined by two factors i.e. space and time. A good algorithm is one that is taking less time and less space, but this is not possible all the time. There is a trade-off between time and space. If you want to reduce the time, then space might increase. Similarly, if you want to reduce the space, then the time may increase. So, you have to compromise with either space or time. Let's learn more about space and time complexity of algorithms.

Asymptotic notations

The word **Asymptotic** means approaching a value or curve arbitrarily closely (i.e., as some sort of limit is taken).

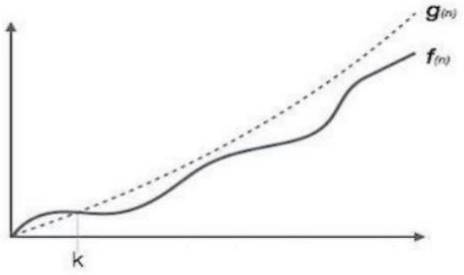
Asymptotic notations are used to write fastest and slowest possible running time for an algorithm.

These are also referred to as 'best case' and 'worst case' scenarios respectively.

"In asymptotic notations, we derive the complexity concerning the size of the input. (Example in terms of n)"' "These notations are important because without expanding the cost of running the algorithm, we can estimate the complexity of the algorithms."

Big-oh-notation (O): It provides possibly asymptotically tight upper bound per f(n) and it does not give best case complexity but can give worst case complexity.

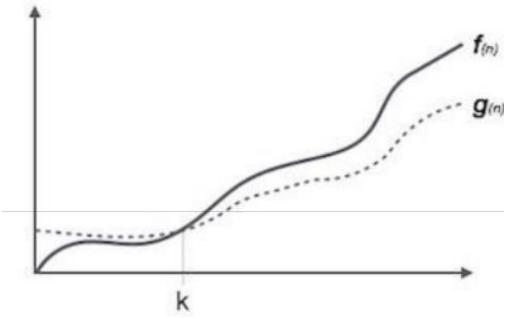
Let f be a nonnegative function. Then we define the three most common asymptotic bounds as fallows. We say that f(n) is Big-oh of g(n) written as f(n) = O(g(n)). iff there are positive constant c and n_0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$.



If f(n) = O(g(n)), we say that g(n) is an upper bound on f(n).

Big- Omega Notation(Ω): It provides possibly asymptotically tight lower bound for f(n) and it does not give worst case complexity but can give best case complexity.

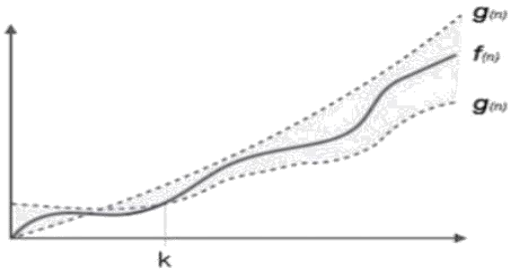
f(n) is said to be big-Omega of g(n) written as f(n)= Ω(g(n)), if there are positive constant c and n_0, such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0$.



If f(n)= Ω (g(n)). We say that g(n) is a lower bound on f(n).

Big Theta Notation (Θ): We say that f(n) is Big Theta of g(n), written as f(n) = Θ (g(n)). If there are positive constants, c1,c2 and n0 such that $0 \leq c1g(n) \leq f(n) \leq c2g(n)$ for all $n \geq n_0$.

Equivalently f(n) = Θ(g(n)_ if and only if f(n) = O (g(n)) and f(n) = Ω (g(n)). If f(n) = Θ(g(n)) we say that g(n) s a tight bound on f(n).



A BRIEF DESCRIPTION OF DATA STRUCTURES

1.8.1 Array

The simplest type of data structure is a linear (or one dimensional) array. A list of a finite number *n* of similar data referenced respectively by a set of *n* consecutive numbers, usually 1, 2, 3 . . . . . *n*. if we choose the name **A** for the array, then the elements of **A** are denoted by subscript notation

A 1, **A** 2, **A** 3 . . . . **A** n

or by the parenthesis notation

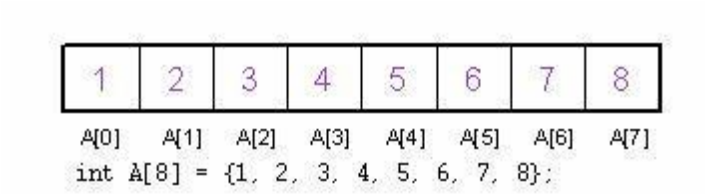
A (1), A (2), A (3) . . . . . A (n)

or by the bracket notation

A [1], A [2], A [3] . . . . . A [n]

Example:

A linear array **A[8]** consisting of numbers is pictured in following figure.

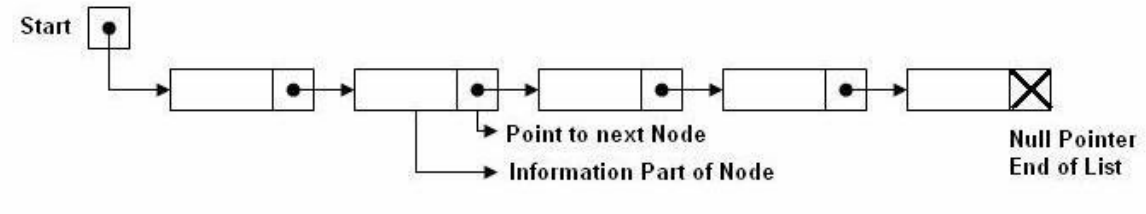


1.8.2 Linked List

A linked list or one way list is a linear collection of data elements, called nodes, where the linear order is given by means of pointers. Each node is divided into two parts:

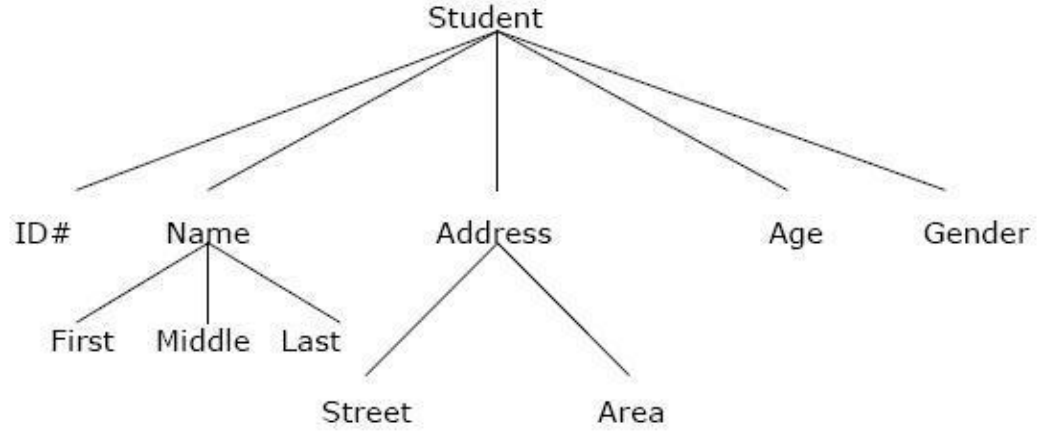
- The first part contains the information of the element/node
- The second part contains the address of the next node (link /next pointer field) in the list. There is a special pointer Start/List contains the address of first node in the list. If this special pointer contains null, means that List is empty.

Example:



1.8.3 Tree

Data frequently contain a hierarchical relationship between various elements. The data structure which reflects this relationship is called a rooted tree graph or, simply, a tree.



1.8.4 Graph

Data sometimes contains a relationship between pairs of elements which is not necessarily hierarchical in nature, e.g. an airline flights only between the cities connected by lines. This data structure is called Graph.

1.8.5 Queue

A queue, also called FIFO system, is a linear list in which deletions can take place only at one end of the list, the Font of the list and insertion can take place only at the other end Rear.

1.8.6 Stack

It is an ordered group of homogeneous items of elements. Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack). The last element to be added is the first to be removed (LIFO: Last In, First Out).

1.9 DATA STRUCTURES OPERATIONS

The data appearing in our data structures are processed by means of certain operations. In fact, the particular data structure that one chooses for a given situation depends largely in the frequency with which specific operations are performed.

The following four operations play a major role in this text:

- **Traversing:** accessing each record/node exactly once so that certain items in the record may be processed. (This accessing and processing is sometimes called "visiting" the record.)
- **Searching:** Finding the location of the desired node with a given key value, or finding the locations of all such nodes which satisfy one or more conditions.
- **Inserting:** Adding a new node/record to the structure.

Deleting: Removing a node/record from the structure

Array

An array is a list of a finite number *'n'* of homogeneous data element such that The elements of the array are reference respectively by an index set consisting of n consecutive numbers.

The element of the array are respectively in successive memory locations. The number *n* of elements is called the length or size of the array. The length or the numbers of elements of the array can be obtained from the index set by the formula When LB = 0,

Length = UB – LB + 1

When LB = 1,

Length = UB

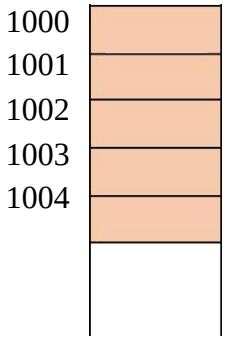
Where,

UB is the largest index called the Upper Bound

LB is the smallest index, called the Lower Bound

Representation of linear arrays in memory

Let LA be a linear array in the memory of the computer. The memory of the computer is simply a sequence of address location as shown below,



LOC (LA [K]) = address of the element LA [K] of the array LA. The elements of LA are stored in successive memory cells.

The computer does not keep track of the address of every element of LA, but needs to keep track only the address of the first element of LA denoted by, LA[0]

Using the base address of LA, the computer calculates the address of any element of LA by the formula

LOC (LA[K]) = *Base* Address + w (Size of elements)(K – lower bound)
Where, w is the number of words per memory cell for the array LA.

ARRAY OPERATIONS

1. Traversing

Let A be a collection of data elements stored in the memory of the computer. Suppose if the contents of the each elements of array A needs to be printed or to count the numbers of elements of A with a given property can be accomplished by Traversing.

Traversing is a accessing and processing each element in the array exactly once.

Algorithm 1: (Traversing a Linear Array)

Hear LA is a linear array with the lower bound LB and upper bound UB. This algorithm traverses LA applying an operation PROCESS to each element of LA using while loop.

1. [Initialize Counter] set K:= LB
2. Repeat step 3 and 4 while K ≤ UB
3. [Visit element] Apply PROCESS to LA [K]
4. [Increase counter] Set K:= K + 1
   [End of step 2 loop]
5. Exit Program

Program:

```
#include<stdio.h>
#include<conio.h> int
main()
```

```
{
int A[100],K=0,UB;
printf("Enter the Array size less than 100: ");
printf("Enter the elements in array: \n");
for(K=0;K<UB;K++)
{
scanf("%d",&A[K]);
}
printf("The Traverse of array is:\n");
for(K=0;K<UB;K++)
{
printf("%d\n",A[K]);
}
getch();
return 0;
}
```

2. Inserting

- Let A be a collection of data elements stored in the memory of the computer.
- Inserting refers to the operation of adding another element to the collection A.
- Inserting an element at the "end" of the linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element.
- Inserting an element in the middle of the array, then on average, half of the elements must be moved downwards to new locations to accommodate the new element and keep the order of the other elements.

Algorithm:

INSERT (LA, N, K, ITEM)
Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$.
This algorithm inserts an element ITEM into the K^{th} position in LA.

1. [Initialize counter]set J:= N
2. Repeat step 3 and 4 while $J \geq K$
3. [Move J^{th} element downward] Set LA [J+1] := LA[J]
4. [Decrease counter] set J:= J – 1
   [End of step 2 loop]
5. [Insert element] set LA[K]:= ITEM
6. [Reset N] set N:= N+1
7. Exit

Program:

```
#include <stdio.h>
int main()
{
  int array[100], position, c, n, value; printf("Enter
  number of elements in array\n"); scanf("%d", &n);
  printf("Enter %d elements\n", n); for (c = 0; c < n; c++)

    scanf("%d", &array[c]);
   printf("Enter the location where you wish to insert an element\n");
  scanf("%d", &position);
   printf("Enter the value to insert\n");
  scanf("%d", &value);
   for (c = n - 1; c >= position - 1; c--)
```

```
    array[c+1] = array[c];
   array[position-1] = value;
   printf("Resultant array is\n");
   for (c = 0; c <= n; c++)
     printf("%d\n", array[c]);
   return 0;
}.
```

Deleting

- Deleting refers to the operation of removing one element to the collection A.
- Deleting an element at the "end" of the linear array can be easily done with difficulties.
- If element at the middle of the array needs to be deleted, then each subsequent elements be moved one location upward to fill up the array.

Algorithm

DELETE (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that K ≤ N. this algorithm deletes the K^{th} element from LA

1. Set ITEM:= LA[K]
2. Repeat for J = K to N – 1
   [Move J + 1 element upward] set LA[J]:= LA[J+1]
   [End of loop]
3. [Reset the number N of elements in LA] set N:= N – 1
4. Exit

program

```
#include <stdio.h>
int main()
{
  int array[100], position, c, n;
   printf("Enter number of elements in array\n"); canf("%d",
   &n);+ for (c = 0; c < n; c++)
     scanf("%d", &array[c]);
   printf("Enter the location where you wish to delete
   element\n"); scanf("%d", &position);
   if (position >= n+1)
     printf("Deletion not possible.\n");
  else
  {
     for (c = position - 1; c < n - 1; c++)
       array[c] = array[c+1];
     printf("Resultant array:\n");
     for (c = 0; c < n - 1; c++)
       printf("%d\n", array[c]);
  }
   return 0;
}
```

Multidimensional Arrays

In C, we can define multidimensional arrays in simple words as array of arrays. Data in multidimensional arrays are stored in tabular form (in row-major order). General form of declaring N-dimensional arrays:

data_type array_name[size1][size2]....[sizeN];
data_type: Type of data to be stored in the array. Here data_type is valid C data type
array_name: Name of the array
size1, size2,... ,sizeN: Sizes of the dimensions

Examples:

Two dimensional array: int two_d[10][20];
Three dimensional array: int three_d[10][20][30];

Size of multidimensional arrays

Total number of elements that can be stored in a multidimensional array can be calculated by multiplying the size of all the dimensions. For example:
The array **int x[10][20]** can store total (10*20) = 200 elements.
Similarly array **int x[5][10][20]** can store total (5*10*20) = 1000 elements.

Example : Sum of two matrices using Two dimensional arrays

```
// C program to find the sum of two matrices of order
2*2 #include <stdio.h>
int main()
{
float a[2][2], b[2][2],
c[2][2]; int i, j;
// Taking input using nested for loop
printf("Enter elements of 1st matrix\n");
for(i=0; i<2; ++i)
for(j=0; j<2; ++j)
{
printf("Enter a%d%d: ", i+1, j+1);
 scanf("%f", &a[i][j]);
}

//Taking input using nested for loop
printf("Enter elements of 2nd matrix\n");
for(i=0; i<2; ++i)
for(j=0; j<2; ++j)
{
printf("Enter b%d%d: ", i+1, j+1);
scanf("%f", &b[i][j]);
}
adding corresponding elements of two arrays
for(i=0; i<2; ++i)
for(j=0; j<2; ++j)
{
c[i][j] = a[i][j] + b[i][j];
}
```

```
//Displaying the sum
printf("\nSum Of Matrix:");
for(i=0; i<2; ++i)
for(j=0; j<2; ++j)
{
printf("%.1f\t", c[i][j]);
if(j==1)
printf("\n");
}
return 0;
}
```

Ouput:

```
Enter elements of 1st matrix
Enter a11: 2;
Enter a12: 0.5;
Enter a21: -1.1;
Enter a22: 2;
Enter elements of 2nd matrix
Enter b11: 0.2;
Enter b12: 0;
Enter b21: 0.23;
Enter b22: 23;
Sum Of Matrix:
2.2     0.5
-0.9    25.0
```

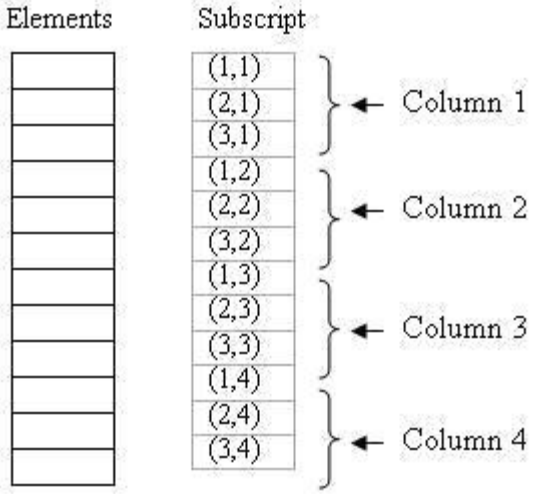
Representation of 2D arrays in memory

A 2D array's elements are stored in continuous memory locations. It can be represented in memory using any of the following two ways:

1. Column-Major Order
2. Row-Major Order

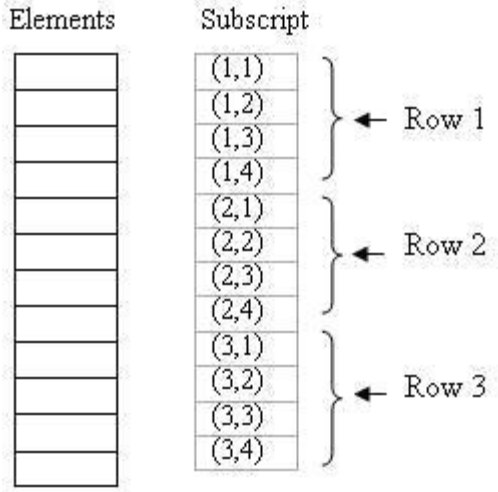
1. Column-Major Order:

In this method the elements are stored column wise, i.e. m elements of first column are stored in first m locations, m elements of second column are stored in next m locations and so on. E.g. A 3 x 4 array will stored as below:



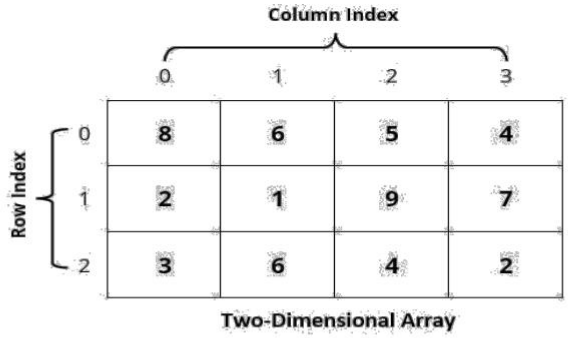
2. Row-Major Order:

In this method the elements are stored row wise, i.e. n elements of first row are stored in first n locations, n elements of second row are stored in next n locations and so on. E.g. A 3 x 4 array will stored as below:

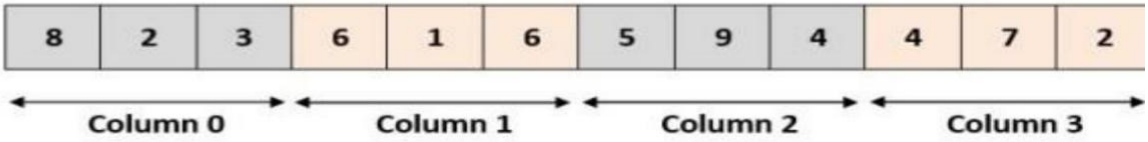


Address Calculation in Double (Two) Dimensional Array:

While storing the elements of a 2-D array in memory, these are allocated contiguous memory locations. Therefore, a 2-D array must be linearized so as to enable their storage. There are two alternatives to achieve linearization: Row-Major and Column-Major.



Column-Major (Column Wise Arrangement)



Address of an element of any array say "A[I][J]" is calculated in two forms as given:

(1) Row Major System (2) Column Major System

Row Major System:

The address of a location in Row Major System is calculated using the following formula:
Address of A [I][J] = BA + W * [N * (I – Lr) + (J – Lc)]

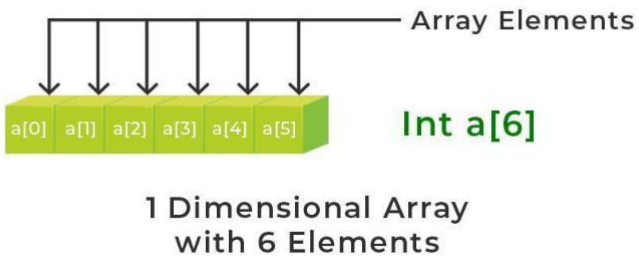
Column Major System:

The address of a location in Column Major System is calculated using the following formula: Address of A [I][J] Column Major Wise = BA + W * [(I – Lr) + M * (J – Lc)] Where,

B = Base address
I = Row subscript of element whose address is to be found
J = Column subscript of element whose address is to be found W = Storage Size of one element stored in the array (in byte)
Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)
Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero) M = Number of rows of the given matrix
N = Number of columns of the given matrix.

Calculation of address of element of 1-D, 2-D, and 3-D using row-major and column-major order

Calculating the address of any element In the 1-D array:
A 1-dimensional array (or single-dimension array) is a type of linear array. Accessing its elements involves a single subscript that can either represent a row or column index.
Example:



1-D array

To find the address of an element in an array the following formula is used-**Address of A[I] = B A+ W * (I – LB)**
I = Subset of element whose address to be found,
B = Base address,
W = Storage size of one element store in any array(in byte),
LB = Lower Limit/Lower Bound of subscript(If not specified assume zero).
Example: Given the base address of an array **A[1300 …… 1900]** as **1020** and the size of each element is 2 bytes in the memory, find the address of **A[1700].**
Solution:
Given:
Base address B = 1020
Lower Limit/Lower Bound of subscript LB = 1300 Storage size of one element store in any array W = 2 Byte Subset of element whose address to be found I = 1700
Formula used:
Address of A[I] = BA + W * (I – LB)
Solution:
Address of A[1700] = 1020 + 2 * (1700 – 1300)
= 1020 + 2 * (400)
= 1020 + 800
Address of A[1700] = 1820

Calculate the address of any element in the 2-D array:
The 2-dimensional array can be defined as an array of arrays. The 2-Dimensional arrays are organized as matrices which can be represented as the collection of rows and columns as array[M][N] where M is the number of rows and N is the number of columns.
Example:

Columns

2D Array	0	1	2
0	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]
2	a[2][0]	a[2][1]	a[2][2]

Rows

2-D array

To find the address of any element in a **2-Dimensional** array there are the following two ways-

1. Row Major Order
2. Column Major Order

1. Row Major Order:
Row major ordering assigns successive elements, moving across the rows and then down the next row, to successive memory locations. In simple language, the elements of an array are stored in a Row-Wise fashion. To find the address of the element using row-major order uses the following formula:
Address of A[I][J] = BA + W * ((I – LR) * N + (J – LC))
I = Row Subset of an element whose address to be found,
J = Column Subset of an element whose address to be found, B = Base address,
W = Storage size of one element store in an array(in byte),
LR = Lower Limit of row/start row index of the matrix(If not given assume it as zero),
LC = Lower Limit of column/start column index of the matrix(If not given assume it as zero), N = Number of column given in the matrix.

Example: Given an array, **arr[1……10][1……15]** with base value **100** and the size of each element is **1 Byte** in memory. Find the address of **arr[8][6]** with the help of row-major order.
Solution:
Given:
Base address B = 100
Storage size of one element store in any array W = 1 Bytes
Row Subset of an element whose address to be found I = 8
Column Subset of an element whose address to be found J = 6 Lower Limit of row/start row index of matrix LR = 1
Lower Limit of column/start column index of matrix = 1
Number of column given in the matrix N = Upper Bound – Lower Bound + 1
= 15–1+1
= 15
Formula:
Address of A[I][J] = B + W * ((I – LR) * N + (J – LC))
Solution:
Address of A[8][6] = 100 + 1 * ((8 – 1) * 15 + (6 – 1))
=100+1*((7)*15+(5))

= 100 + 1 * (110)
Address of A[I][J] = 210

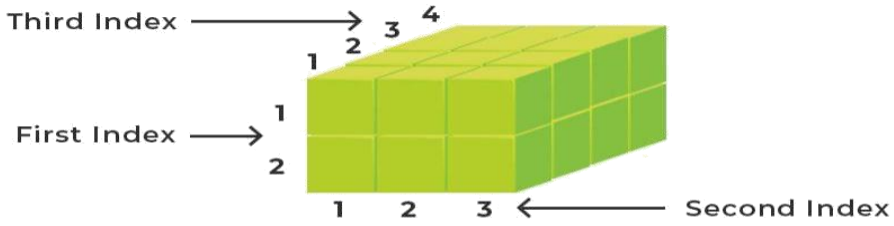
2. Column Major Order:
If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order. To find the address of the element using column-major order use the following formula:
Address of A[I][J] = B + W * ((J – LC) * M + (I – LR)) I
= Row Subset of an element whose address to be found,
J = Column Subset of an element whose address to be
found, B = Base address,
W = Storage size of one element store in any array(in byte),
LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),
LC = Lower Limit of column/start column index of matrix(If not given assume it as
zero), M = Number of rows given in the matrix.

Example: Given an array **arr[1……10][1……15]** with a base value of **100** and the size of each element is **1 Byte** in memory find the address of arr[8][6] with the help of column-major order.
Solution:
Given:
Base address B = 100
Storage size of one element store in any array W = 1 Bytes
Row Subset of an element whose address to be found I = 8
Column Subset of an element whose address to be found J =
6 Lower Limit of row/start row index of matrix LR = 1
Lower Limit of column/start column index of matrix = 1
Number of Rows given in the matrix M = Upper Bound – Lower Bound + 1
= 10–1+1
= 10
Formula: used
Address of A[I][J] = BA + W * ((J – L_C) * M + (I – L_R))
Address of A[8][6] = 100 + 1 * ((6 – 1) * 10 + (8 – 1))
= 100 + 1 * ((5) * 10 + (7))
= 100 + 1 * (57)
Address of A[I][J] = 157
From the above examples, it can be observed that for the same position two different address locations are obtained that's because in row-major order movement is done across the rows and then down to the next row, and in column-major order, first move down to the first column and then next column. So both the answers are right.
So it's all based on the position of the element whose address is to be found for some cases the same answers is also obtained with row-major order and column-major order and for some cases, different answers are obtained.

Calculate the address of any element in the 3-D Array:
A **3-Dimensional** array is a collection of 2-Dimensional arrays. It is specified by using three subscripts:

1. Block size
2. Row size
3. Column size

More dimensions in an array mean more data can be stored in that array.
Example:



Three-Dimensional Array
with 24 Elements

3-D array

To find the address of any element in 3-Dimensional arrays there are the following two ways-

- Row Major Order
- Column Major Order
-

1. Row Major Order:

To find the address of the element using row-major order, use the following formula:

Address of A[i][j][k] = BA + W *(P * N * (i-x) +P *(j-y) + (k-z))

Here:

B = Base Address (start address)
W = Weight (storage size of one element stored in the array)
M = Row (total number of rows)
N = Column (total number of columns)
P = Width (total number of cells depth-wise)
x = Lower Bound of Row
y = Lower Bound of Column
z = Lower Bound of Width

Example: Given an array, **arr[1:9, -4:1, 5:10]** with a base value of **400** and the size of each element is **2 Bytes** in memory find the address of element **arr[5][-1][8]** with the help of row-major order?

Solution: Given:

Block Subset of an element whose address to be found I = 5
Row Subset of an element whose address to be found J = -1
Column Subset of an element whose address to be found K = 8
Base address B = 400
Storage size of one element store in any array(in Byte) W = 2
Lower Limit of blocks in matrix x = 1
Lower Limit of row/start row index of matrix y = -4
Lower Limit of column/start column index of matrix z = 5
N(Column) = Upper Bound – Lower Bound + 1 = 1 – (-4) + 1 = 6
P(depth)= Upper Bound – Lower Bound + 1 = 10 – 5 + 1 = 6

Formula used:

Address of A[i][j][k] = BA + W *(P * N * (i-x) +P *(j-y) + (k-z))

Solution:

Address of arr[5][-1][8] = 400 + 2 * {[6 * 6 * (5 – 1)] + 6 * [(-1 + 4)]} + [8 – 5]
= 400 + 2 * (6*6*4)+(6*3)+3
= 400 + 2 * (165)
= 730

2. Column Major Order:

To find the address of the element using column-major order, use the following formula:1 **Address of A[i][j][k]= BA + W(M * P(k – z) + M *(j – y) + (i-x))**

Here:
B = Base Address (start address)
W = Weight (storage size of one element stored in the array)
M = Row (total number of rows)
N = Column (total number of columns)
P = Width (total number of cells depth-wise)
x = Lower Bound of block (first subscipt)
y = Lower Bound of Row
z = Lower Bound of Column

Example: Given an array **arr[1:8, -5:5, -10:5]** with a base value of **400** and the size of each element is **4 Bytes** in memory find the address of element **arr[3][3][3]** with the help of column-major order?
Solution:
Given:
Row Subset of an element whose address to be found I = 3
Column Subset of an element whose address to be found J = 3
Block Subset of an element whose address to be found K = 3
Base address B = 400
Storage size of one element store in any array(in Byte) W = 4
Lower Limit of blocks in matrix x = 1
Lower Limit of row/start row index of matrix y = -5
Lower Limit of column/start column index of matrix z = -10
M (row)= Upper Bound – Lower Bound + 1 = 8 -1+1 = 8
P (depth)= Upper Bound – Lower Bound + 1 = 5 + 10 + 1 = 16
Formula used:
Address of A[i][j][k]= BA + W(M * P(k – z) + M *(j – y) + (i-x))
Solution:
Address of arr[3][3][3] = 400 + 4 * ((16*8*(3-1)+8*(3-(-5)+(3-(-10)))
= 400 + 4 * ((128*2 + 8*8 + 13)
= 400 + 4 * (333) = 400 + 1332 = 1732

Calculate the address of any element in the N-D Array:

For a given subscript K, the effective index E, of L, is the number of indices preceding K, in the index set, and E, can be calculated from EK, lower bound

Then the address LOC(C(K1, K2..... KN) of an arbitrary element of C can be obtained from the formula (column-major order)

$$LOC(C(K1, K2..... KN) = Base(C) + w[(((... (E_N L_{N-1} + E_{N-1}) L_{N-2}) + E_3) L_2 + E_2) L_1 + E_1] \quad (4.8)$$

or from the formula (row-major order)

$$LOC(C(K1, K2..... KN) = Base(C) + w[(...((E_1 L_2 + E_2) L_3 + E_3) L_4 + ... + E_{N-1}) L_N + E_N] \quad (4.9)$$

according to whether C is stored in column-major or row-major order. Once again, Base(C) denotes the address of the first element of C, and w denotes the number of words per memory location.

Example: Suppose a three-dimensional array MAZE is declared using MAZE(2:8, -4:1, 6:10) Then the lengths of the three dimensions of MAZE are, respectively. $L_1 = 8 - 2 + 1 = 7$, $L_2 = 1+4+1 = 6$, L3 = 10-6+1 =5
Accordingly, MAZE contains $L_1 * L_2 * L_3 = 7 * 6 * 5 = 210$ elements.
Suppose the programming language stures MAZE in memory in row-major order, and suppose Base(MAZE) = 200 and there are w = 4 words per memory cell. The address of an element of MAZE-for example, MAZE[5, -1, 8]-is obtained as follows. The effective indices of the subscripts are, respectively, $E_1 = 5 - 2 = 3$, E2 = -1+4 = 3, E3 = 8 - 6 = 2
Using Eq. (4.9) for row-major order, we have:

Therefore, $E_1 * L_2 = 3*6 = 18$
$E_1 * L_2 + E_2 = 18 + 3 = 21$
$(E_1 * L_2 + E_2) * L_3 = 21 * 5 = 105$
$(E_1 * L_2 + E_2) * L_3 + E_3 = 105 + 2 = 107$
Therefore,
LoC(MAZE[5, - 1, 8]) = 200 + 4*(107) = 200 + 428 = 628

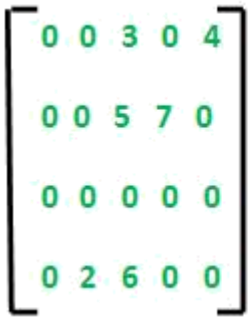
Sparse Matrix and its representations

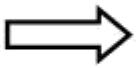
A matrix• is a two-dimensional data object made of m rows and n columns, therefore having total m x n values. If most of the elements of the matrix have **0 value**, then it is called a sparse matrix.

Why to use Sparse Matrix instead of simple matrix ?

- **Storage:** There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
- **Computing time:** Computing time can be saved by logically designing a data structure traversing only non-zero elements.

Example:
00304
00570
00000
02600





Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeroes in the matrix are of no use in most of the cases. So, instead of storing zeroes with non-zero elements, we only store non-zero elements. This means storing non-zero elements with **triples- (Row, Column, value).** Sparse Matrix Representations can be done in many ways following are two common representations:

1. Array representation
2. Linked list representation

1: Using Arrays:

2D array is used to represent a sparse matrix in which there are three rows named as

- **Row:** Index of row, where non-zero element is located
- **Column:** Index of column, where non-zero element is located
- **Value:** Value of the non zero element located at index – (row,column)

Implementation:

```
// C++ program for Sparse Matrix Representation
// using Array
#include<stdio.h>

int main()
{
   // Assume 4x5 sparse matrix int
   sparseMatrix[4][5] =
   {
      {0,0,3,0,4},
      {0,0,5,7,0},
      {0,0,0,0,0}, {0,2,6,0,0}
```

```
};

int size = 0;
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 5; j++)
        if (sparseMatrix[i][j] != 0)
            size++;

// number of columns in compact Matrix (size) must be
// equal to number of non - zero elements in
// sparse Matrix
int compactMatrix[3][size];

// Making of new matrix int k
= 0;
for (int i = 0; i < 4; i++) for (int j
    = 0; j < 5; j++)
        if (sparseMatrix[i][j] != 0)
        {
            compactMatrix[0][k] = i; compactMatrix[1][k]
            = j; compactMatrix[2][k] = sparseMatrix[i][j];
            k++;
        }

for (int i=0; i<3; i++)
{
    for (int j=0; j<size; j++)
        printf("%d ", compactMatrix[i][j]);

    printf("\n");
  }
  return 0;
}
```

Function for Dynamic memory allocation:

The functions **malloc()** and **calloc()** are library functions that allocate memory dynamically. Dynamic means the memory is allocated during runtime (execution of the program) from the heap segment.

Initialization

malloc() allocates a memory block of given size (in bytes) and returns a pointer to the beginning of the block. malloc() doesn't initialize the allocated memory. If you try to read from the allocated memory without first initializing it, then you will invoke undefined behavior, which usually means the values you read will be garbage values

c**alloc()** allocates the memory and also initializes every byte in the allocated memory to 0. If you try to read the value of the allocated memory without initializing it, you'll get 0 as it has already been initialized to 0 by calloc().

Parameters

malloc() takes a single argument, which is the **number of bytes to allocate.**
Unlike malloc(), calloc() takes two arguments:

1. Number of blocks to be allocated.
2. Size of each block in bytes.

Return Value

After successful allocation in malloc() and calloc(), a pointer to the block of memory is returned otherwise NULL is returned which indicates failure.

Difference between malloc() and calloc() in C

S.No.	malloc()	calloc()
1	malloc() is a function that creates one block of memory of a fixed size.	calloc() is a function that assigns a specified number of blocks of memory to a single variable
2	malloc() only takes one argument	calloc() takes two arguments.
3	malloc() is faster than calloc.	calloc() is slower than malloc()
4	malloc() has high time efficiency	calloc() has low time efficiency
5	malloc() is used to indicate memory allocation	calloc() is used to indicate contiguous memory allocation
6	malloc() does not initialize the memory to zero	calloc() initializes the memory to zero
7	malloc() does not add any extra memory overhead	calloc() adds some extra memory overhead

Linked List

A linked list is a non-sequential collection of data items. It is a dynamic data structure. For every data item in a linked list, there is an associated pointer that would give the memory location of the next data item in the linked list.
The data items in the linked list are not in consecutive memory locations. They may be anywhere, but the accessing of these data items is easier as each data item contains the address of the next data item.

Advantages of linked lists:
Linked lists have many advantages. Some of the very important advantages are:
1. Linked lists are dynamic data structures. i.e., they can grow or shrink during the execution of a program.
2. Linked lists have efficient memory utilization. Here, memory is not pre- allocated. Memory is allocated whenever it is required and it is de- allocated (removed) when it is no longer needed.
3. Insertion and Deletions are easier and efficient. Linked lists provide flexibility in inserting a data item at a specified position and deletion of the data item from the given position.
4. Many complex applications can be easily carried out with linked lists.

Disadvantages of linked lists:
1. It consumes more space because every node requires a additional pointer to store address of the next node.
2. Searching a particular element in list is difficult and also time consuming.

Types of Linked Lists:
Basically we can put linked lists into the following four items:
1. Single Linked List.
2. Double Linked List.
3. Circular Linked List.

Comparison between array and linked list:

ARRAY	LINKED LIST
Size of an array is fixed	Size of a list is not fixed
Memory is allocated from stack	Memory is allocated from heap
It is necessary to specify the number of elements during declaration (i.e., during compile time).	It is not necessary to specify the number of elements during declaration (i.e., memory is allocated during run time).
It occupies less memory than a linked list for the same number of elements.	It occupies more memory.
Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.	Inserting a new element at any position can be carried out easily.
Deleting an element from an array is not possible.	Deleting an element is possible.

Single linked list is one in which all nodes are linked together in some sequential manne r. Hence, it is also called as linear linked list.

A **double linked list** is one in which all nodes are linked togethe r by multiple links which helps in accessing both the successor node (next node) and predeces sor node (previous node) from any arbitrary node within the list. Therefore each node in a double linked list has two link fields (pointers) to point to the left node (previous) and the right node (next). This helps to traverse in forward direction and backward direction.

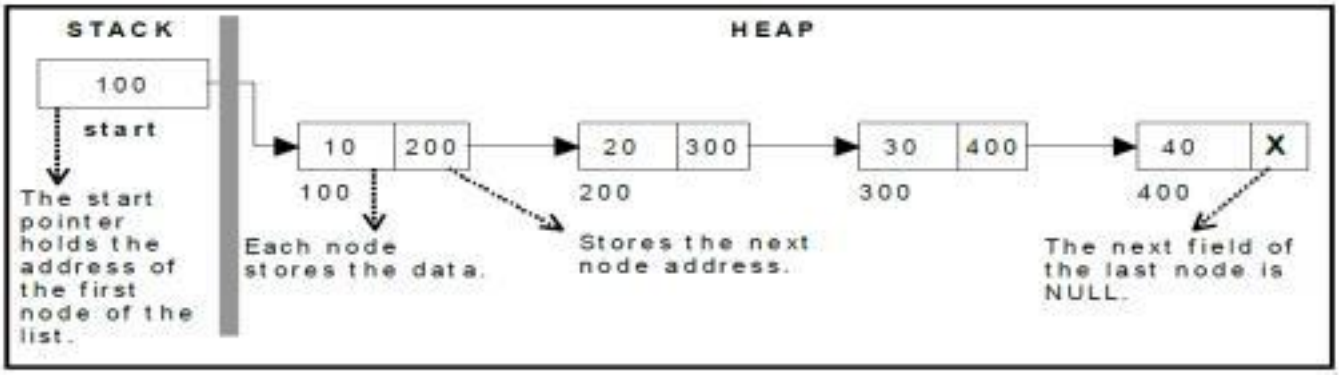
A **circular linked list** is one, which has no beginning and no end. A single linked list can be made a circular linked list by simply storing address of the very first node in the link field of the last node.

Single Linked List:

A linked list allocates space for each element separately in its own block of memory called a "node". The list gets an overall structure by using pointers to connect all its nodes together like the links in a chain.

Each node contains two fields; a "data" field to store what eve r element, and a "next" field which is a pointer used to link to the next node.

Each node is allocated in the heap using malloc(), so the node memory continues to exist until it is explicitly de-allocated using free(). The front of the list is a pointer to the “start” node. A single linked list is shown in figure.



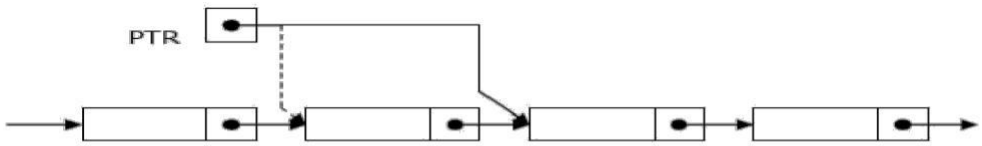
The beginning of the linked list is stored in a " **start** " pointer which points to the first node. The first node contains a pointer to the second node.
The second node contains a pointer to the third node, ... and so on. The last node in the list has its next field set to NULL to mark the end of the list. Code can access any node in the list by starting at the **start** and following the next pointers .
The **start** pointer is an ordinary local pointe r variable, so it is drawn separately on the left top to show that it is in the stack. The list nodes are drawn on the right to show that they are allocated in the heap.
The basic operations in a single linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing.

Traversing a Linked List

Suppose we want to traverse LIST in order to process each node exactly once. The traversing algorithm uses a pointer variable PTR which points to the node that is currently being processed. Accordingly, PTR->NEXT points to the next node to be processed so,

PTR=HEAD [Moves the pointer to the first node of the list] PTR=PTR->NEXT [Moves the p
ointer to the next node in the list.]



Algorithm: (Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of list. The variable PTR point to the node currently being processed.

1. Set PTR=HEAD. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR!=NULL.
3. Apply PROCESS to PTR-> INFO.
4. Set PTR= PTR-> NEXT [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

Program

```
    void display()
1.  {
2.      struct node *ptr;
3.      ptr = head;
4.      if(ptr == NULL)
5.          printf("Nothing to print");
6.       else
7.      {
8.         printf("\nprinting values . . . . .\n");
9.         while (ptr!=NULL)
10.        {
11.           printf("\n%d",ptr->data);
12.           ptr = ptr -> next;
13.        } }}
```

Searching a Linked List:

Let list be a linked list in the memory and a specific ITEM of information is given to search. If ITEM is actually a key value and we are searching through a LIST for the record containing ITEM, then ITEM can appear only once in the LIST. Search for wanted ITEM in List can be performed by traversing the list using a pointer variable PTR and comparing ITEM with the contents PTR->INFO of each node, one by one of list.

Algorithm: SEARCH(INFO, NEXT, HEAD, ITEM, PREV, CURR, SCAN)
LIST is a linked list in the memory. This algorithm finds the location LOC of the node where ITEM first appear in LIST, otherwise sets LOC=NULL.

1. Set PTR=HEAD.
2. Repeat Step 3 and 4 while PTR≠NULL:
3. if ITEM = PTR->INFO then:
   Set LOC=PTR, and return. [Search is successful.]
   [End of If structure.]
4. Set PTR=PTR->NEXT
   [End of Step 2 loop.]
5. Set LOC=NULL, and return. [Search is unsuccessful.]
6. Exit.

Program

```
    void search()
1.  {
2.      struct node *ptr;
3.      int item,i=0,flag;
4.      ptr = head;
5.      if(ptr == NULL)
6.      {
7.         printf("\nEmpty List\n");
8.      }
9.      else
10.     {
11.        printf("\nEnter item which you want to search?\n");
12.        scanf("%d",&item);
13.        while (ptr!=NULL)
14.        {
15.           if(ptr->data == item)
16.           {
```

```
17.             printf("item found at location %d ",i+1);
18.             flag=0;
19.          }
20.          else
21.          {
22.             flag=1;
23.          }
24.          i++;
25.          ptr = ptr -> next;
26.       }
27.       if(flag==1)
28.       {
29.          printf("Item not found\n");
30.       }   }   }
```

Creation of a node:

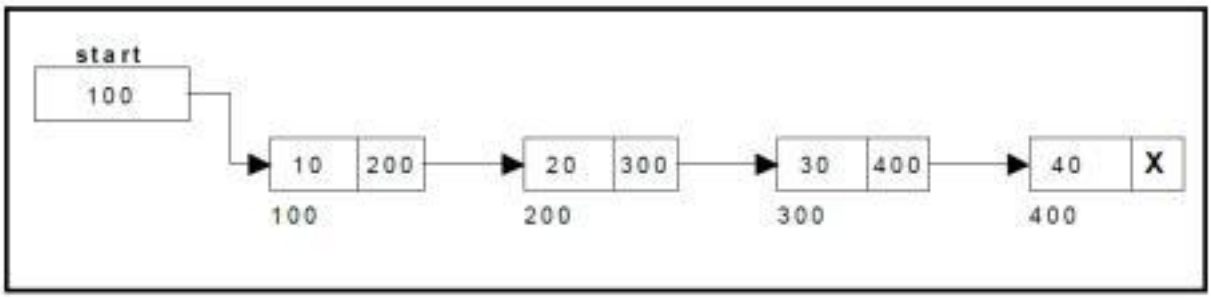
Creating a singly linked list starts with creating a node. Sufficient memory has to be allocated for creating a node. The information is stored in the memory, allocated by using the malloc() function. after allocating memory for the structure of type node, the information for the item (i.e., data) has to be read from the user, set next field to NULL and finally returns the address of the node.

Creating a Singly Linked List with 'n' number of nodes:

The following steps are to be followed to create 'n' number of nodes:

- Get the new node using malloc();
- If the list is empty, assign new node as start. start = newnode;
- If the list is not empty, follow the steps given below:
- The next field of the new node is made to point the first node (i.e. start node) in the list by assigning the address of the first node.
- The start pointer is made to point the new node by assigning the address of the new node.
- Repeat the above steps 'n' times.

Figure shows 4 items in a single linked list stored at different locations in memory.



Insertion of a Node:

One of the most primitive operations that can be done in a singly linked list is the insertion of a node. Memory is to be allocated for the new node (in a similar way that is done while creating a list) before reading the data. The new node will contain empty data field and empty next field. The data field of the new node is then stored with the information read from the user. The next field of the new node is assigned to NULL. The new node can then be inserted at three different places namely:

- ➢ Inserting a node at the beginning.
- ➢ Inserting a node at the end.
- ➢ Inserting a node at intermediate position.

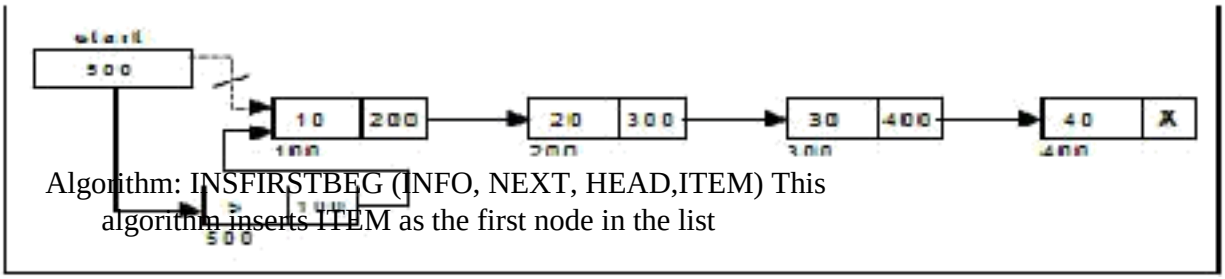
Inserting a node at the beginning:

The following steps are to be followed to insert a new node at the beginning of the list:

- ➢ Get the new node using malloc().
- ➢ If the list is empty then *start = newnode.*
- ➢ If the list is not empty, follow the steps given below:

```
newnode -> next = start;
start = newnode;
```

Figure shows inserting a node into the single linked list at the beginning.



Algorithm: INSFIRSTBEG (INFO, NEXT, HEAD,ITEM) This algorithm inserts ITEM as the first node in the list

Step 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 7
[END OF IF]
Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT
Step 3: SET NEW_NODE = PTR
Step 4 NEW_NODE → INFO = ITEM
Step 5: : SET NEW_NODE → NEXT = HEAD
Step 6: SET HEAD = NEW_NODE
Step 7: EXIT

Program

```
    void beginsert()
1.  {
2.      struct node *ptr;
3.      int item;
4.      ptr = (struct node *) malloc(sizeof(struct node *));
5.      if(ptr == NULL)
6.      {
7.          printf("\nOVERFLOW");
8.      }
9.      else
10.     {
11.         printf("\nEnter value\n");
12.         scanf("%d",&item);
13.         ptr->data = item;
14.         ptr->next = head;
15.         head = ptr;
16.         printf("\nNode inserted");
17.     } }
```

Inserting a node at the end:

The following steps are followed to insert a new node at the end of the list:

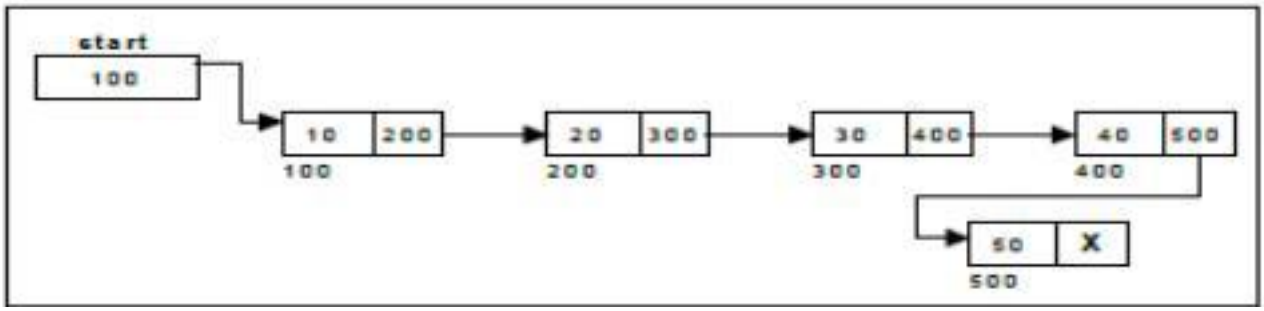
- Get the new node using malloc()
- If the list is empty then *start = newnode*.
- If the list is not empty follow the steps given below:

```
temp = star t ;
while(temp -> next != NULL)
temp = temp -> next;
temp -> next = newnode;
```

INSERTEND(INFO, NEXT, HEAD,ITEM)

Step 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 9
[END OF IF]
Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT
Step 3: SET NEW_NODE = PTR
Step 4: NEW_NODE → INFO = ITEM
Step 5: : SET NEW_NODE → NEXT = NULL
Step 6: SET P =HEAD
Step 7: Repeat step 8 while P→NEXT!= NULL
Step 8: SET P=P→ NEXT
[END OF LOOP]
Step 8: SET P - > NEXT = NEW_NODE
Step 9: EXIT

Figure shows inserting a node into the single linked list at the end.



Program:

```
    void lastinsert()
1.  {
2.      struct node *ptr,*P;
3.      int item;
4.      ptr = (struct node*)malloc(sizeof(struct node));
5.      if(ptr == NULL)
6.      {
7.          printf("\nOVERFLOW");
8.      }
9.      else
10.     {
11.         printf("\nEnter value?\n");

12.         scanf("%d",&item);
13.         ptr->data = item;
14.         ptr->next=NULL;
15.         if(head == NULL)
16.         {
17.             head = ptr;
18.             printf("\nNode inserted");
19.         }
20.         else
```

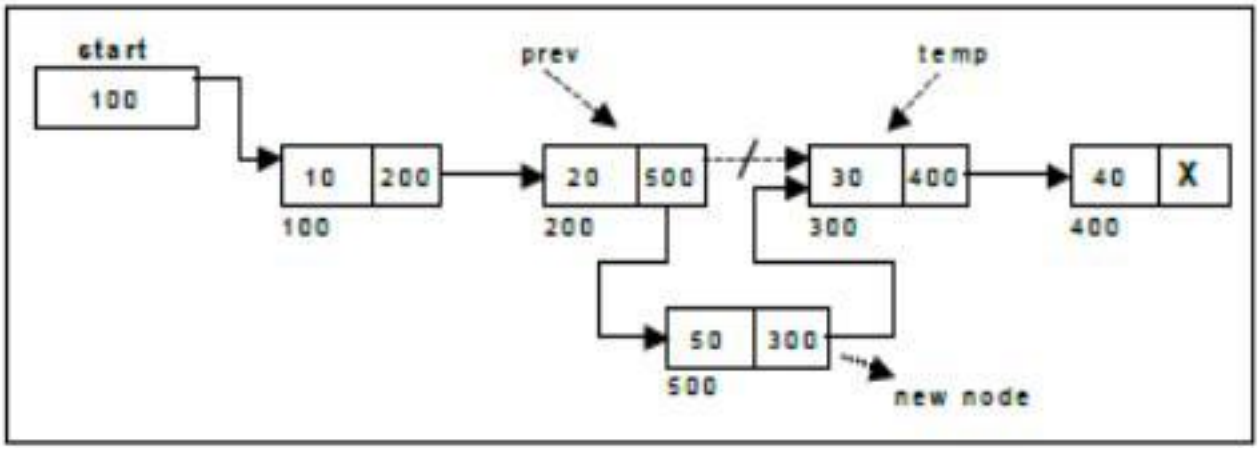
```
21.        {
22.            P = head;
23.            while (P -> next != NULL)
24.            {
25.                P =P -> next;
26.            }
27.            P->next = ptr;
28.            printf("\nNode inserted");
29.
31.        }
32.     }
33. }
```

Inserting a node at ANY position:

The following steps are followed, to insert a new node in an intermediate position in the list:

- Get the new node using malloc().
- Ensure that the specified position is in between first node and last node. If not, specified position is invalid..
- Store the starting address (which is in start pointer) in temp and prev pointers . Then traverse the temp pointer upto the specified position followed by prev pointer .
- After reaching the specified position, follow the steps given below:
  prev -> next = newnode;
  newnode -> next = temp;
- Let the intermediate position be 3.

.Figure shows inserting a node into the single linked list at a specified intermediate position other than beginning and end.



INSERTATLOC(INFO, NEXT, HEAD,ITEM, LOC)

Step1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 13
[END OF IF]

Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT

STEP 3: SET NEW_NODE = PTR

STEP 4 :SET NEW_NODE → INFO = ITEM

STEP 5: SET TEMP = HEAD

STEP 6: SET I = 0
STEP 7: REPEAT STEP 8 to 10 UNTIL I<loc-1
STEP 8: TEMP1= TEMP
TEMP = TEMP → NEXT
STEP 9: IF TEMP = NULL
WRITE "DESIRED NODE NOT PRESENT"
GOTO STEP 13
END OF IF
STEP 10: SET I = I+1
END OF LOOP
STEP 11: NEW_NODE → NEXT = TEMP1 → NEXT
STEP 12: TEMP1 → NEXT = NEW_NODE
STEP 13: EXIT

Program:

```
    void randominsert()
1.  {
2.      int i,loc,item;
3.      struct node *ptr, *temp;
4.      ptr = (struct node *) malloc (sizeof(struct node));
5.      if(ptr == NULL)
6.      {
7.          printf("\nOVERFLOW");
8.      }
9.      else
10.     {
11.         printf("\nEnter element value");
12.         scanf("%d",&item);
13.         ptr->data = item;
14.         printf("\nEnter the location after which you want to insert ");
15.         scanf("\n%d",&loc);
16.         temp=head;
17.         for(i=0;i<loc;i++)
18.         {
19.             temp = temp->next;
20.             if(temp == NULL)
21.             {
22.                 printf("\ncan't insert\n");
23.                 return;
24.             }
25.        }
26.         ptr ->next = temp ->next;
27.         temp ->next = ptr;
28.         printf("\nNode inserted");
29.     }
30. }
```

Deletion of a node:

Another primitive operation that can be done in a singly linked list is the deletion of a node. Memory is to be released for the node to be deleted. A node can be deleted from the list from three different places namely.

- Deleting a node at the beginning.
- Deleting a node at the end.
- Deleting a node at intermediate position.

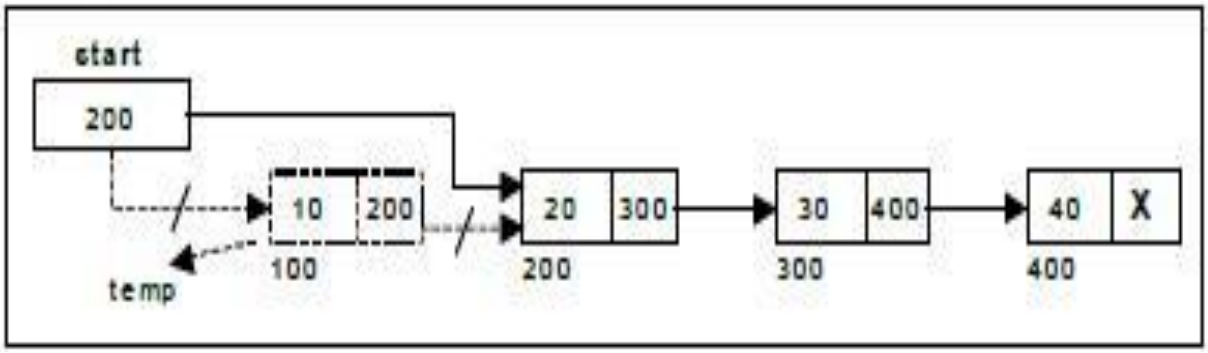
Deleting a node at the beginning:

The following steps are followed, to delete a node at the beginning of the list:

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given below:

temp = star t ;
star t = star t -> next;
free(temp);

Figure shows deleting a node at the beginning of a single linked list.



Algorithm

Step 1: IF HEAD = NULL
Write UNDERFLOW
Go to Step 6
[END OF IF]
Step 2: SET PTR = HEAD
Step 3: SET HEAD = HEAD -> NEXT
Step 4: PTR->NEXT=AVAIL
Step 5 AVAIL=PTR
Step 6: EXIT

Program:

```
void begin_delete()
1.  {
2.      struct node *ptr;
3.      if(head == NULL)
4.      {
5.          printf("\nList is empty\n");
6.      }
7.      else
8.      {
9.          ptr = head;
10.         head = ptr->next;
11.         free(ptr);
12.         printf("\nNode deleted from the begining ...\n");
13.     } }
```

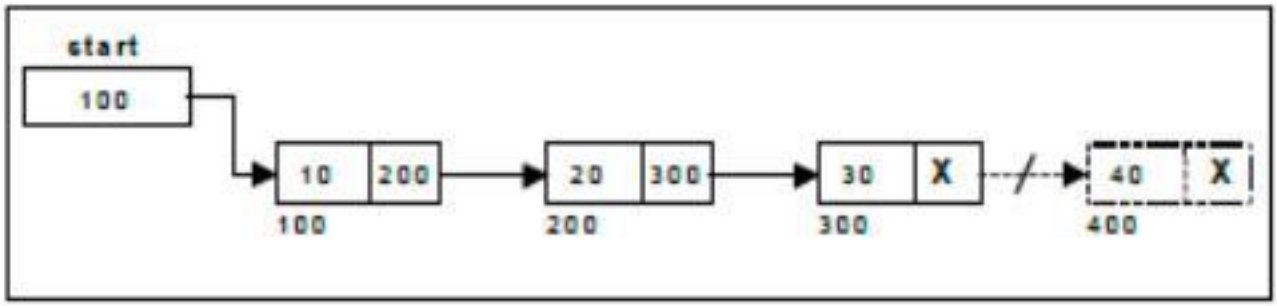
Deleting a node at the end:

The following steps are followed to delete a node at the end of the list:

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given below:

```
temp = prev = start ;
while(temp -> next != NULL)
{
prev = temp;
temp = temp -> next;
}
prev -> next =
NULL; free(temp);
```

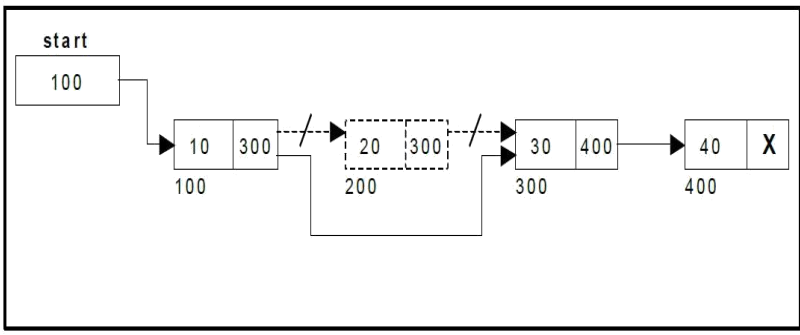
Figure shows deleting a node at the end of a single linked list.



Algorithm

Step 1: IF HEAD = NULL
Write UNDERFLOW
Go to Step 9
[END OF IF]
Step 2: SET PTR = HEAD
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT!= NULL
Step 4: SET TEMP = PTR
Step 5: SET PTR = PTR -> NEXT

[END OF LOOP]
Step 6: SET TEMP -> NEXT = NULL
Step 7: PTR->NEXT=AVAIL
Step 8: AVAIL=PTR
Step 9: EXIT



Program:

```
     void last_delete()
1.   {
2.      struct node *ptr,*ptr1;
3.      if(head == NULL)
4.            printf("\nlist is empty");
5.      else if(head -> next == NULL)
6.      {
7.         head = NULL;
8.         free(head);
9.         printf("\nOnly node of the list deleted ...\n");
10.     }
11.     else
12.     {
13.        ptr = head;
14.        while(ptr->next != NULL)
15.        {
16.           ptr1 = ptr;
17.           ptr = ptr ->next;
18.        }
19.        ptr1->next = NULL;
20.        free(ptr);
21.        printf("\nDeleted Node from the last ...\n");
22.     } }
```

Deleting a node at specified position:

The following steps are followed, to delete a node from an intermediate position in the list (List must contain more than two node).

- If list is empty then display 'Empty List' message
- If the list is not empty, follow the steps given below.

Figure shows deleting a node at a specified intermediate position other than beginning and end from a single linked list.

Algorithm

STEP 1: IF HEAD = NULL
WRITE UNDERFLOW
GOTO STEP 12
END OF IF
STEP 2: SET TEMP = HEAD
STEP 3: SET I = 0
STEP 4: REPEAT STEP 5 TO 8 UNTIL I<loc-1
STEP 5: TEMP1 = TEMP
STEP 6: TEMP = TEMP → NEXT
STEP 7: IF TEMP == NULL
WRITE "DESIRED NODE NOT PRESENT"
GOTO STEP 12
END OF IF
STEP 8: I = I+1
END OF LOOP
STEP 9: TEMP1 → NEXT = TEMP → NEXT
STEP 10: PTR->NEXT=AVAIL
Step 11: AVAIL=PTR
STEP 12: EXIT

Proogram:

```
void random_delete()
{
   struct node *ptr,*ptr1;
   int loc,i;
   printf("\n Enter the location of the node after which you want to perform deletion \n");
   scanf("%d",&loc);
   ptr=head;
   for(i=0;i<loc-1;i++)
   {
      ptr1 = ptr;
      ptr = ptr->next;

      if(ptr == NULL)
      {
         printf("\nCan't delete");
         return;
      }
   }
   ptr1 ->next = ptr ->next;
   free(ptr);
   printf("\nDeleted node %d ",loc+1);
}
```

A complete Linked List program

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node *head=NULL;

void beginsert ();
void lastinsert ();
void randominsert();
void begin_delete();
void last_delete() ;
void random_delete();
void display();
void search();
void main ()
{
   int choice =0;
   while(choice != 9)
   {
      printf("\n\n*********Main Menu*********\n"); printf("\nChoose one option from the
      following list ...\n");
      printf("\n===============================================\n");
      printf("\n1.Insert in begining\n2.Insert at last\n3.Insert at any random location\n4.Delete
           from Beginning\n5.Delete from last\n6.Delete node after specified location\n7.Search for an
           element\n8.Show\n9.Exit\n");
      printf("\nEnter your choice?\n");
      scanf("\n%d",&choice);
      switch(choice)
      {
```

```
            case 1:
            beginsert();
            break;
            case 2:
            lastinsert();
            break;
            case 3:
            randominsert();
            break;
            case 4:
            begin_delete();
            break;
            case 5:
            last_delete();
            break;
            case 6:
            random_delete();
            break;
            case 7:
            search();
            break;
            case 8:
            display();
            break;
            case 9:
            exit(0);
            break;
            default:
            printf("Please enter valid choice..");
        }
    }
}
void beginsert()
{
    struct node *ptr;
    int item;
    ptr = (struct node *) malloc(sizeof(struct node *));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter value\n");
        scanf("%d",&item);
        ptr->data = item;
        ptr->next = head;
        head = ptr;
        printf("\nNode inserted");
    } }
void lastinsert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
```

```
        printf("\nEnter value?\n");
        scanf("%d",&item);
        ptr->data = item;
        if(head == NULL)
        {
           ptr -> next = NULL;
           head = ptr;
           printf("\nNode inserted");
        }
        else
        {
           temp = head;
           while (temp -> next != NULL)
           {
              temp = temp -> next;
           }
           temp->next = ptr;
           ptr->next = NULL;
           printf("\nNode inserted");

        }
 }
  }

void randominsert()
{
   int i,loc,item;
   struct node *ptr, *temp;
   ptr = (struct node *) malloc (sizeof(struct node));
   if(ptr == NULL)
   {
      printf("\nOVERFLOW");
   }
   else
   {
      printf("\nEnter element value");
      scanf("%d",&item);
      ptr->data = item;
      printf("\nEnter the location after which you want to insert ");
      scanf("\n%d",&loc);
      temp=head;
      for(i=0;i<loc;i++)
      {
         temp = temp->next;
         if(temp == NULL)
         {
            printf("\ncan't insert\n");
            return;
         }
      }
      ptr ->next = temp ->next;
      temp ->next = ptr;
      printf("\nNode inserted");
   }
}
void begin_delete()
{
   struct node *ptr;
   if(head == NULL)
   {
      printf("\nList is empty\n");
```

```
    }
    else
    {
        ptr = head;
        head = ptr->next;
        free(ptr);
        printf("\nNode deleted from the begining ...\n");
    }
}
void last_delete()
{
    struct node *ptr,*ptr1;
    if(head == NULL)
    {
        printf("\nlist is empty");
    }
    else if(head -> next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nOnly node of the list deleted ...\n");
    }
    else
    {
        ptr = head;
        while(ptr->next != NULL)
        {
            ptr1 = ptr;
            ptr = ptr ->next;
        }
        ptr1->next = NULL;
        free(ptr);
        printf("\nDeleted Node from the last ...\n");
    }
}
void random_delete()
{
    struct node *ptr,*ptr1;
    int loc,i;
    printf("\n Enter the location of the node after which you want to perform deletion \n");
    scanf("%d",&loc);
    ptr=head;
    for(i=0;i<loc;i++)
    {
        ptr1 = ptr;
        ptr = ptr->next;

        if(ptr == NULL)
        {
            printf("\nCan't delete");
            return;
        }
    }
    ptr1 ->next = ptr ->next;
    free(ptr);
    printf("\nDeleted node %d ",loc+1);
}
void search()
{
    struct node *ptr;
    int item,i=0,flag;
```

```
    ptr = head;
    if(ptr == NULL)
    {
       printf("\nEmpty List\n");
    }
    else
    {
       printf("\nEnter item which you want to search?\n");
       scanf("%d",&item);
       while (ptr!=NULL)
       {
          if(ptr->data == item)
          {
             printf("item found at location %d ",i+1);
             flag=0;
          }
          else
          {
             flag=1;
          }
          i++;
          ptr = ptr -> next;
       }
       if(flag==1)
       {
          printf("Item not found\n");
       }
    }
  }
void display()
{
    struct node *ptr;
    ptr = head;
    if(ptr == NULL)
    {
       printf("Nothing to print");
    }
    else
    {
       printf("\nprinting values . . . . .\n");
       while (ptr!=NULL)
       {
          printf("\n%d",ptr->data);
          ptr = ptr -> next;
       }
    }
}
```

Doubly Linked List:

A doubly linked list is a two-way list in which all nodes will have two links. This helps in accessing both successor node and predecessor node from the given node position. It provides bi-directional traversing. Each node contains three fields:

- Left link.
- Data.
- Right link.

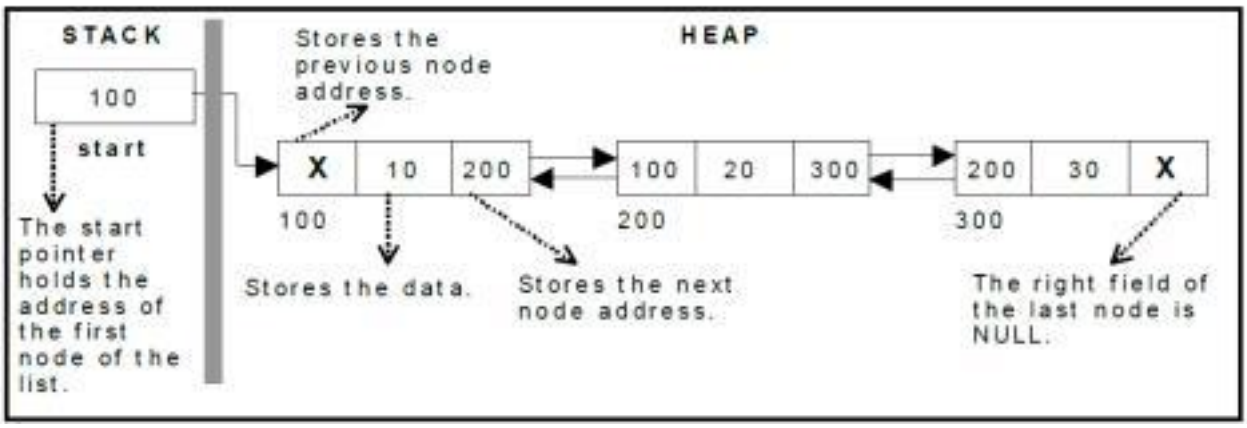
The left link point s to the predecessor node and the right link points to the successor node. The data field store s the required data.

Many applications require searching forward and backward thru nodes of a list. For example searching for a name in a telephone directory would need forward and backward scanning thru a region of the whole list.

The basic operations in a double linked list are:

- Creation.
- Insertion.
- Deletion.
- Traver sing.

A double linked list is shown in figure.



The beginning of the double linked list is stored in a " **start** " pointe r which points to the first node. The first node's left link and last node's right link is set to NULL. The following code gives the structure definition:

```
struct dlinklist
{
struct dlinklist * left ;
int d at a;
struct dlinklist * right ;
} ;
```

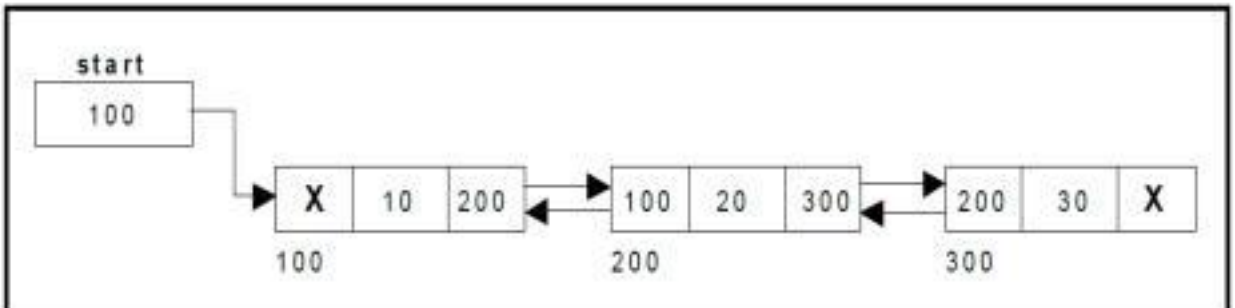
Creating a node for Doubly Linked List:

Creating a double linked list starts with creating a node. Sufficient memory has to be allocated for creating a node. The information is stored in the memory, allocated by using the malloc() function. after allocating memory for the structure of type node, the information for the item (i.e., data) has to be read from the user and set left field to NULL and right field also set to NULL **Creating a Doubly Linked List with 'n' number of nodes :**

The following steps are to be followed to create 'n' number of nodes:

- Get the new node using malloc().
- If the list is empty then *start = newnode*.
- If the list is not empty, follow the steps given below:
- The left field of the new node is made to point the previous node.
- The previous nodes right field must be assigned with address of the new node.
- Repeat the above steps 'n' times.

Figure shows 3 items in a double linked list stored at different locations.



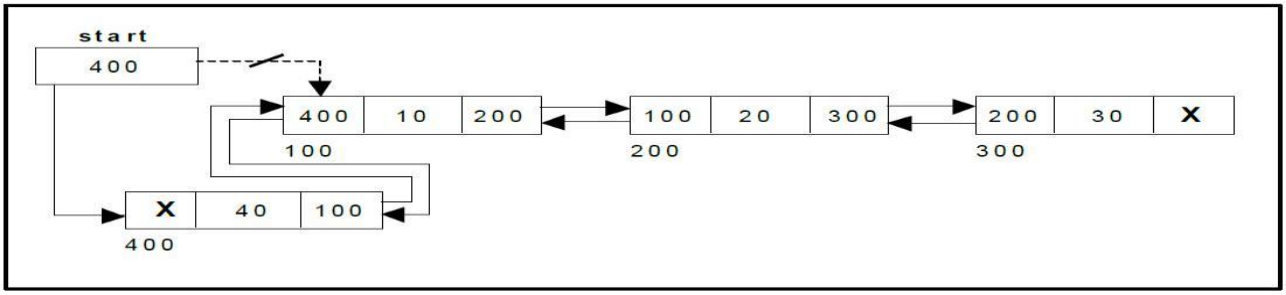
Insertion:

Insertion of a Node at the beginning: The following steps are to be followed to insert a new node at the beginning of the list:

- Get the new node using malloc().
- If the list is empty then start = newnode.
- If the list is not empty, follow the steps given

below: newnode -> right = start;
start -> left = newnode;
start = newnode;

Figure shows inserting a node into the double linked list at the beginning.



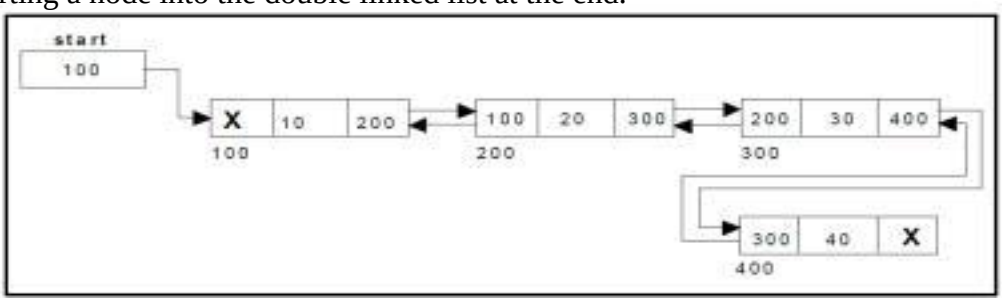
Step 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 8
[END OF IF]
Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT
AVAIL->PREV=NULL
Step 2: SET NEW_NODE = PTR
Step 3 SET NEW_NODE → INFO = ITEM
Step 4: SET NEW_NODE → NEXT = HEAD
Step 5: SET NEW_NODE → PREV =NULL
Step 6: SET HEAD → PREV = NEW_NODE
Step 7: SET HEAD = NEW_NODE
Step 8: EXIT

Inserting a node at the end: The following steps are followed to insert a new node at the end of the list:

- Get the new node using malloc().
- If the list is empty then start = newnode.
- If the list is not empty follow the steps given

below: temp = start;
while(tem p -> right != NULL)
temp = temp -> right;
temp -> right = newnode;
newnode -> left = temp;

Figure shows inserting a node into the double linked list at the end.



ALGORITHM

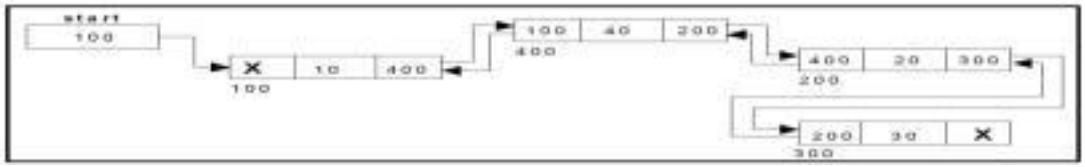
Step 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 11
[END OF IF]
Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT
AVAIL->PREV=NULL
Step 2: SET NEW_NODE = PTR
Step 3 SET NEW_NODE → INFO = ITEM
Step 4: : SET NEW_NODE → NEXT = NULL
Step 5: SET NEW_NODE → PREV = NULL
Step 6: SET TEMP =HEAD
Step 7: Repeat step 8 while TEMP→NEXT!= NULL
Step 8: SET TEMP= TEMP→ NEXT
[END OF LOOP]
Step 9: SET TEMP - > NEXT = NEW_NODE
Step 10: SET NEW_NODE - > PREV = TEMP
Step 11: EXIT

Inserting a node at an intermediate position: The following steps are followed, to insert a new node in an intermediate position in the list:

- Get the new node using malloc().
- Ensure that the specified position is in between first node and last node. If not, specified position is invalid.
- Store the starting address (which is in start pointer) in temp and prev pointers. Then traverse the temp pointer upto the specified position followed by prev pointer.
- After reaching the specified position, follow the steps given below:

```
newnode -> left = temp;
newnode -> right = temp -> right;
temp -> right -> left = newnode;
temp -> right = newnode;
```

Figure shows inserting a node into the double linked list at a specified intermediate position other than beginning and end.



ALGORITHM

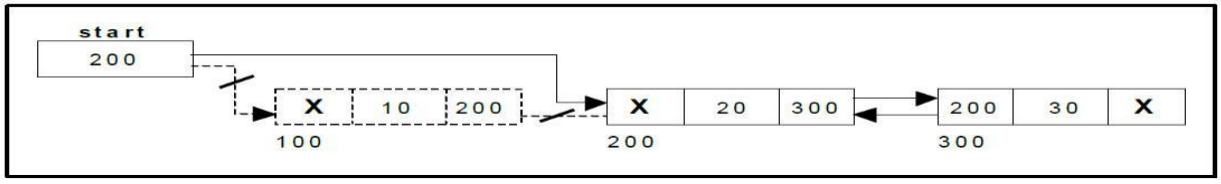
STEP 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 16
[END OF IF]
Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT
AVAIL->PREV=NULL
STEP 2: SET NEW_NODE = PTR
STEP 3: SET NEW_NODE → INFO = ITEM
STEP **4: :** SET NEW_NODE → NEXT = NULL
STEP **5:** SET NEW_NODE → PREV = NULL
STEP 6: SET TEMP = HEAD
STEP 7: SET I = 0
STEP 8: REPEAT STEP 9 TO 11 UNTIL I<loc-1
STEP 9 SET TEMP1= TEMP
TEMP = TEMP → NEXT
STEP 10: IF TEMP = NULL
WRITE "DESIRED NODE NOT PRESENT"
RETURN
END OF IF
STEP 11: SET I = I+1
END OF LOOP
STEP 12: SET TEMP1 → NEXT → PREV = NEW_NODE
STEP 13: SET NEW_NODE → NEXT = TEMP1 → NEXT
STEP 14: SET TEMP1 → NEXT = NEW_NODE
STEP 15: SET NEW_NODE → PREV = TEMP1
STEP 16: EXIT

Deletion:

Deleting a node at the beginning: The following steps are followed, to delete a node at the beginning of the list: • If list is empty then display 'Empty List' message.

• If the list is not empty, follow the steps given below: temp = start;
start = start -> right;
start -> left =
NULL; free(temp);

Figure shows deleting a node at the beginning of a double linked list.



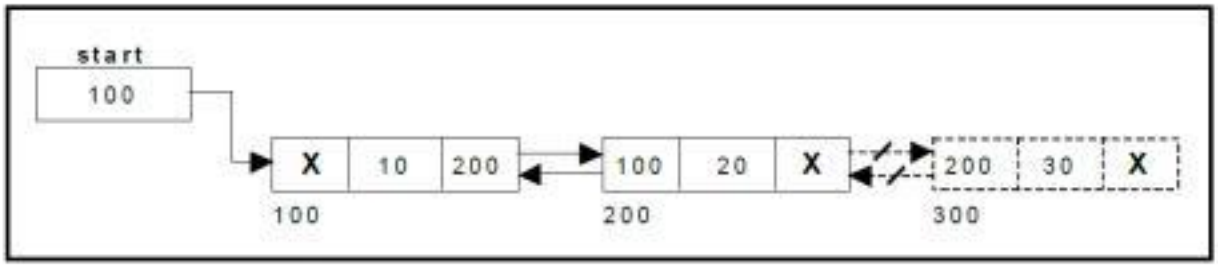
Algorithm

Step 1: IF HEAD = NULL
Write UNDERFLOW
RETURN
[END OF IF]
Step 2: SET PTR = HEAD
Step 3: SET HEAD = HEAD -> NEXT
Step 4: SET HEAD -> PREV = NULL
Step 5: PTR->NEXT=AVAIL
AVAIL-> PREV =PTR
AVAIL=PTR
Step 6: EXIT

Deleting a node at the end: The following steps are followed to delete a node at the end of the list:

- If list is empty then display 'Empty List' message
- If the list is not empty, follow the steps given

below: temp = start;
while(temp -> right !=
NULL) { temp = temp ->
right; } temp -> left ->
right = NULL;
free(temp);
Figure shows deleting a node at the end of a doubly linked list.



ALGORITHM

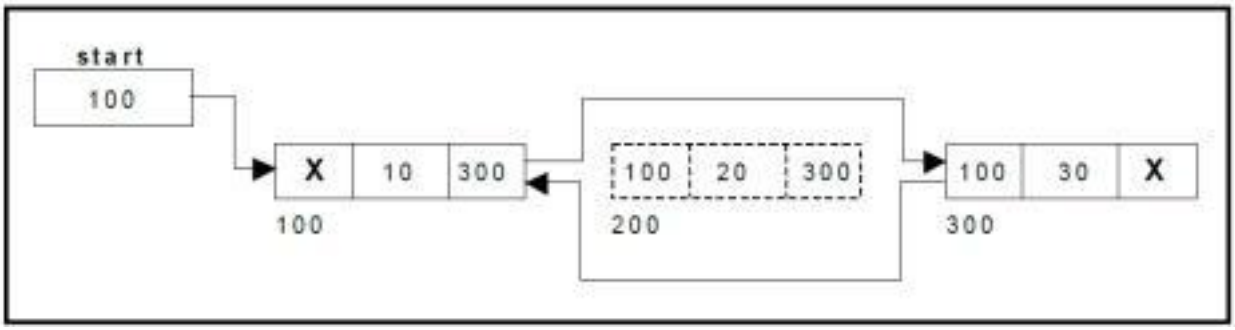
Step 1: IF HEAD = NULL
Write UNDERFLOW
RETURN [END OF IF]
Step 2: SET PTR = HEAD
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT!= NULL
Step 4: SET TEMP = PTR
Step 5: SET PTR = PTR -> NEXT
[END OF LOOP]
Step 6: SET TEMP -> NEXT = NULL**:**
Step 7: FREE PTR
PTR->NEXT=AVAIL
AVAIL-> PREV =PTR
AVAIL=PTR
Step 8: EXIT

Deleting a node at Intermediate position: The following steps are followed, to delete a node from an intermediate position in the list (List must contain more than two nodes).

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given below:
- Get the position of the node to delete.
- Ensure that the specified position is in between first node and last node. If not, specified position is invalid.
- Then perform the following steps:

```
if(pos > 1 && pos < nodectr)
{ temp = start;
 i = 1;
while(i < pos)
{ temp = temp -> right;
i+ +; }
temp -> right -> left = temp -> left;
temp -> left -> right = temp -> right;
free(tem p);
printf("\n node deleted..."); }
```

Figure shows deleting a node at a specified intermediate position other than beginning and end from a doubly linked list.



ALGORITHM

STEP 1: IF HEAD = NULL
WRITE UNDERFLOW
RETURN
[END OF IF]
STEP 2: SET TEMP = HEAD
STEP 3: SET I = 0
STEP 4: REPEAT STEP 5 TO 8 UNTIL I<loc-1
STEP 5: TEMP1 = TEMP
STEP 6: TEMP = TEMP → NEXT
STEP 7: IF TEMP = NULL
WRITE "DESIRED NODE NOT PRESENT"
RETURN
END OF IF
STEP 8: I = I+1
END OF LOOP
STEP 9: TEMP1 → NEXT = TEMP → NEXT
STEP 10: TEMP1 → NEXT →PREV = TEMP1
STEP 11: FREE TEMP
PTR->NEXT=AVAIL
AVAIL-> PREV =PTR
AVAIL=PTR
STEP 12: EXIT

Traversal and displaying a list (Left to Right): To display the information, you have to traverse the list, node by node from the first node, until the end of the list is reached. The following steps are followed, to traverse a list from left to right:

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given

below: temp = start;
while(tem p !=
NULL) { print
temp -> data;
temp = temp ->
right; }

Traversal and displaying a list (Right to Left): To display the information from right to left, you have to traverse the list, node by node from the first node, until the end of the list is reached. The following steps are followed, to traverse a list from right to left:

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given

below: temp = start; while(tem p -> right !=
NULL)
temp = temp ->
right; while(tem p
!= NULL) { print
temp -> data;
temp = temp-
>left;}

Complete Program for Doubly Link list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
   struct node *prev;
   struct node *next;
   int data;
};
struct node *head;
void insertion_beginning();
void insertion_last();
void insertion_specified();
void deletion_beginning();
void deletion_last();
void deletion_specified();
void display();
void search();
void main ()
{
int choice =0;
   while(choice != 9)
   {
```

```
            printf("\n*****Main Menu******\n");
            printf("\nChoose one option from the following list
            ...\n"); printf("\n======\n");
            printf("\n1.Insert in begining\n2.Insert at last\n3.Insert at any random location\n4.Delete
                 from Beginning\n 5.Delete from last\n6.Delete the node after the given
                 data\n7.Search\n8.
                  Show\n9.Exit\n");
            printf("\nEnter your choice?\n");
            scanf("\n%d",&choice);
            switch(choice)
            {
               case 1:
               insertion_beginning();
               break;
               case 2:
                     insertion_last();
               break;
               case 3:
               insertion_specified();
               break;
               case 4:
               deletion_beginning();
               break;
               case 5:
               deletion_last();
               break;
               case 6:
               deletion_specified();
               break;
               case 7:
               search();
               break;
            case 8:
            display();
            break;
            case 9:
            exit(0);
            break;
            default:
            printf("Please enter valid choice..");
         }
      }
}
void insertion_beginning()
{
   struct node *ptr;
   int item;
   ptr = (struct node *)malloc(sizeof(struct node));
   if(ptr == NULL)
   {
```

```
        printf("\nOVERFLOW");
    }
    else
    {
     printf("\nEnter Item value");
     scanf("%d",&item);

    if(head==NULL)
    {
        ptr->next = NULL;
        ptr->prev=NULL;
        ptr->data=item;
        head=ptr;
    }
    else
    {
        ptr->data=item;
        ptr->prev=NULL;
        ptr->next = head;
        head->prev=ptr;
        head=ptr;
    }
    printf("\nNode inserted\n");
}
     }
void insertion_last()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node *) malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter value");

        scanf("%d",&item);
         ptr->data=item;
        if(head == NULL)
        {
            ptr->next = NULL;
            ptr->prev = NULL;
            head = ptr;
        }
        else
        {
          temp = head;
          while(temp->next!=NULL)
```

```
        {
           temp = temp->next;
        }
        temp->next = ptr;
        ptr ->prev=temp;
        ptr->next = NULL;
        }
         }
    printf("\nnode inserted\n");
   }
void insertion_specified()
{
  struct node *ptr,*temp;
  int item,loc,i;
  ptr = (struct node *)malloc(sizeof(struct node));
  if(ptr == NULL)
  {
     printf("\n OVERFLOW");
  }
  else
  {
     temp=head;
     printf("Enter the location");
     scanf("%d",&loc);
     for(i=0;i<loc;i++)
     {
        temp = temp->next;
        if(temp == NULL)
        {
           printf("\n There are less than %d elements", loc);
           return;
        }
     }
     printf("Enter value");
     scanf("%d",&item);
     ptr->data = item;
     ptr->next = temp->next;
     ptr -> prev = temp;
     temp->next = ptr;
     temp->next->prev=ptr;
     printf("\nnode inserted\n");
  }
}
void deletion_beginning()
{
   struct node *ptr;
   if(head == NULL)
   {
      printf("\n UNDERFLOW");
   }
```

```
    else if(head->next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }
    else
    {
        ptr = head;
        head = head -> next;
        head -> prev = NULL;
        free(ptr);
        printf("\nnode deleted\n");
    }
}
void deletion_last()
{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\n UNDERFLOW");
    }
    else if(head->next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }
    else
    {
        ptr = head;
        if(ptr->next != NULL)
        {
            ptr = ptr -> next;
        }
        ptr -> prev -> next = NULL;
        free(ptr);
        printf("\nnode deleted\n");
    }
}
void deletion_specified()
{
    struct node *ptr, *temp;
    int val;
    printf("\n Enter the data after which the node is to be deleted : ");
    scanf("%d", &val);
    ptr = head;
    while(ptr -> data != val)
    ptr = ptr -> next;
    if(ptr -> next == NULL)
```

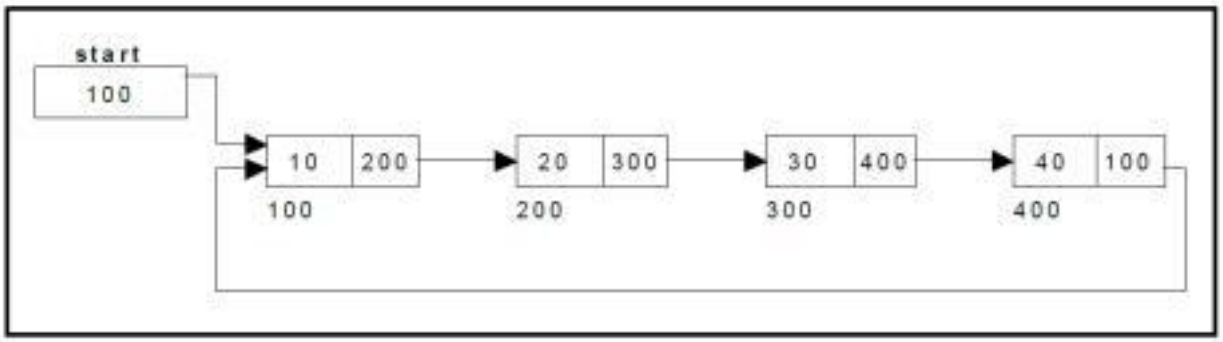
```
    {
        printf("\nCan't delete\n");
    }
    else if(ptr -> next -> next == NULL)
    {
        ptr ->next = NULL;
    }
    else
    {
        temp = ptr -> next;
        ptr -> next = temp -> next;
        temp -> next -> prev = ptr;
        free(temp);
        printf("\nnode deleted\n");
    }
}
void display()
{
    struct node *ptr;
    printf("\n printing values...\n");
    ptr = head;
    while(ptr != NULL)
    {
        printf("%d\n",ptr->data);
        ptr=ptr->next;
    }
}
void search()
{
    struct node *ptr;
    int item,i=0,flag;
    ptr = head;
    if(ptr == NULL)
    {
        printf("\nEmpty List\n");
    }
    else
    {
        printf("\nEnter item which you want to search?\n");
        scanf("%d",&item);
        while (ptr!=NULL)
        {
            if(ptr->data == item)
            {
                printf("\nitem found at location %d ",i+1);
                flag=0;
                break;
            }
            else
            {
```

```
            flag=1;
          }
          i++;
          ptr = ptr -> next;
       }
       if(flag==1)
       {
          printf("\nItem not found\n");
       }
  }    }
```

Circular Single Linked List:

It is just a single linked list in which the link field of the last node points back to the address of the first node. A circular linked list has no beginning and no end. It is necessary to establish a special pointer called start pointer always pointing to the first node of the list.

Circular linked lists are frequently used instead of ordinary linked list because many operations are much easier to implement. In circular linked list no null pointers are used, hence all pointers contain valid address. A circular single linked list is shown in figure .



The basic operations in a circular single linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing.

Creating a circular single Linked List:

he following steps are to be followed to create 'n' number of nodes:

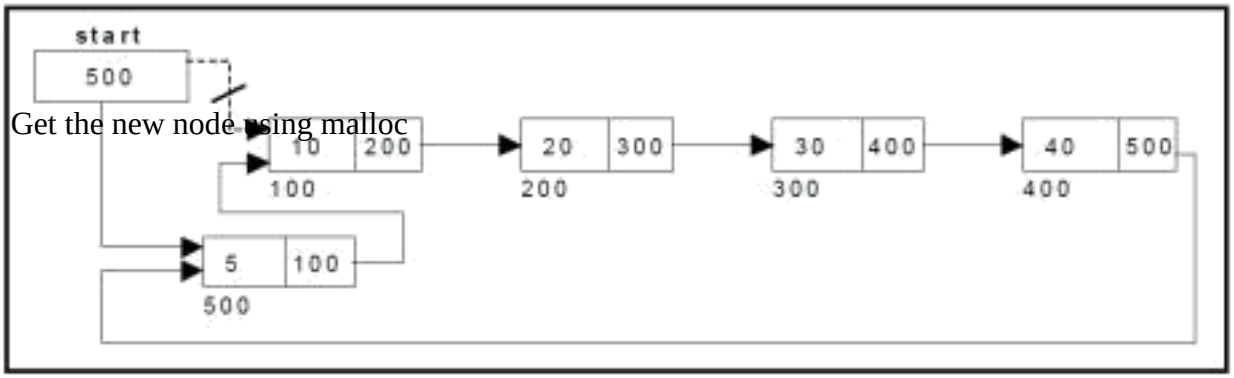
- Get the new node using malloc().
- If the list is empty, assign new node as start. start = newnode;
- If the list is not empty, follow the steps given below:

```
temp = start;
while(tem p -> next != NULL)
temp = temp -> next;
temp -> next = newnode;
```

- Repeat the above steps 'n' times.
- newnode -> next = start;

Inserting a node at the begnning:

The following steps are to be followed to insert a new node at the beginning of the circular list:



• Get the new node using malloc

• If the list is empty, assign new node as start. start = newnode;
newnode -> next = start;

• If the list is not empty, follow the steps given below: last = start;
while(last -> next != start) last = last -> next; newnode -> next = start; start = newnode;
last -> next = start;

Figure shows inserting a node into the circular single linked list at the beginning.

ALGORITHM

Step 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 10
[END OF IF]

Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT

Step 2: SET PTR -> INFO = ITEM
Step 3: SET PTR -> NEXT= NULL
Step 4: SET TEMP = HEAD
Step 5: Repeat Step 8 while TEMP -> NEXT != HEAD
Step 6: SET TEMP = TEMP -> NEXT
[END OF LOOP]
Step 7: SET PTR -> NEXT = HEAD
Step 8: SET TEMP → NEXT = PTR
Step 9: SET HEAD = PTR
Step 10: EXIT

Inserting a node at the end:

The following steps are followed to insert a new node at the end of the list:

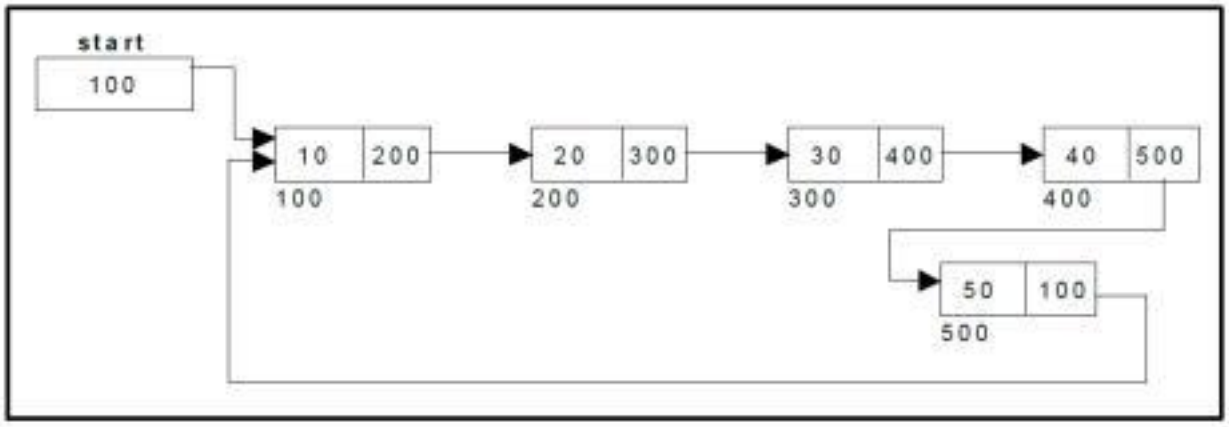
• Get the new node using malloc()

• If the list is empty, assign new node as start. start = newnode;
newnode -> next = start;

• If the list is not empty follow the steps given below: temp = start;

while(tem p -> next !=
start) temp = temp -> next;
temp -> next = newnode;
newnode -> next = start;

Figure shows inserting a node into the circular single linked list at the end.



ALGORITHM

Step 1: IF AVAIL = NULL
Write OVERFLOW
Go to Step 09
[END OF IF]

Step 2: PTR=AVAIL
AVAIL=AVAIL->NEXT

Step 2: SET PTR -> INFO = ITEM

Step 3: SET PTR -> NEXT = NULL

Step 4: SET TEMP = HEAD

Step 5: Repeat Step 8 while TEMP -> NEXT != HEAD

Step 6: SET TEMP = TEMP -> NEXT
[END OF LOOP]

Step 7: SET TEMP -> NEXT = PTR

Step 8: SET PTR -> NEXT=HEAD

Step 9: EXIT

Insertion at random same as insertion at random in singly link list.

Deleting a node at the beginning:

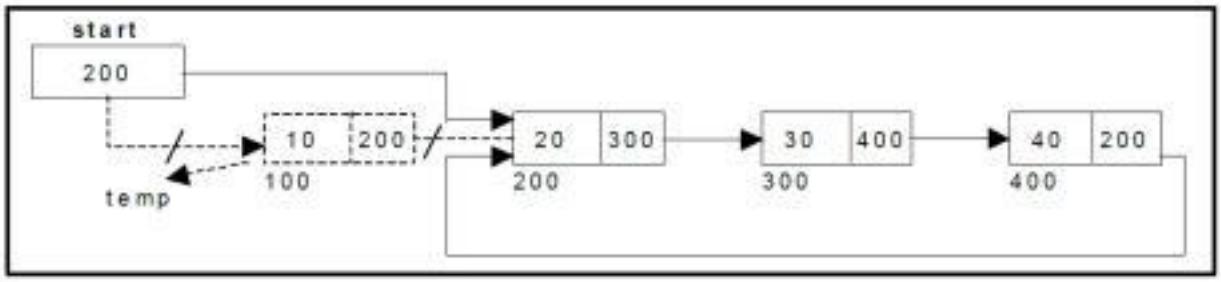
The following steps are followed, to delete a node at the beginning of the list:

- If the list is empty, display a message 'Empty List'.
- If the list is not empty, follow the steps given

below: last = temp = start;
while(last -> next !=
start) last = last -> next;
start = start -> next;
last -> next = start;

- After deleting the node, if the list is empty then start = NULL.

Figure shows deleting a node at the beginning of a circular single linked list.



ALGORITHM

Step 1: IF HEAD = NULL
Write UNDERFLOW
RETURN
[END OF IF]

Step 2: SET PTR = HEAD

Step 3: Repeat Step 4 while PTR → NEXT != HEAD

Step 4: SET PTR = PTR → next
[END OF LOOP]

Step 5: SET PTR → NEXT = HEAD → NEXT

Step 6: FREE HEAD

Step 7: SET HEAD = PTR → NEXT

PTR->NEXT=AVAIL
AVAIL=PTR

Step 8: EXIT

Deleting a node at the end:

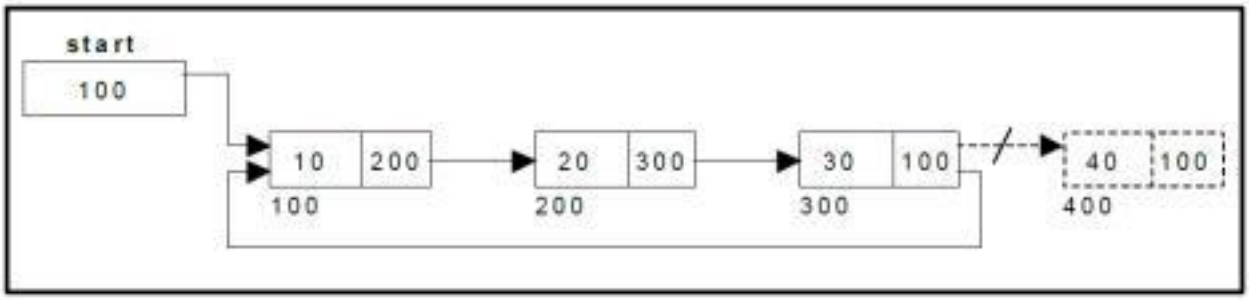
The following steps are followed to delete a node at the end of the list:

- If the list is empty, display a message ‘Empty List’.
- If the list is not empty, follow the steps given

below: temp = start;
prev = start;
while(tem p -> next != start)

{ prev = temp;
temp = temp -> next; }
prev -> next = start;

• After deleting the node, if the list is empty then start = NULL. Figure shows deleting a node at the end of a circular single linked list.



Algorithm

Step 1: IF HEAD = NULL
Write UNDERFLOW
RETURN
[END OF IF]
Step 2: SET PTR = HEAD
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT != HEAD
Step 4: SET TEMP = PTR
Step 5: SET PTR = PTR -> NEXT
[END OF LOOP]
Step 6: SET TEMP -> NEXT = HEAD
Step 7: FREE PTR
PTR->NEXT=AVAIL
AVAIL=PTR

Step 8: EXIT

Deletion at random same as deletion at random in singly link list.

Traversing a circular single linked list from left to right:
The following steps are followed, to traverse a list from left to right:
• If list is empty then display 'Empty List' message.
• If the list is not empty, follow the steps given below: temp = start;
do { printf("%d ", temp -> data); temp = temp -> next; }
while(temp != start);

Complete Program for circular singly Link list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node *head;
void beginsert ();
void lastinsert ();
void randominsert();
void begin_delete();
```

void last_delete();

```
void random_delete();
void display();
void search();
void main ()
{
   int choice =0;
   while(choice != 7)
   {
      printf("\n*********Main Menu*********\n"); printf("\nChoose one option from the
      following list ...\n");
      printf("\n==========================================\n");
      printf("\n1.Insert in begining\n2.Insert at last\n3.Delete from Beginning\n4.Delete from last\
           n5.Search for an element\n6.Show\n7.Exit\n");
      printf("\nEnter your choice?\n");
      scanf("\n%d",&choice);
      switch(choice)
      {
         case 1:
         beginsert();
         break;
         case 2:
         lastinsert();
         break;
         case 3:
         begin_delete();
         break;
         case 4:
         last_delete();
         break;
         case 5:
         search();
         break;
         case 6:
         display();
         break;
         case 7:
         exit(0);
         break;
         default:
         printf("Please enter valid choice..");
      }
   }
}
void beginsert()
{
   struct node *ptr,*temp;
   int item;
   ptr = (struct node *)malloc(sizeof(struct node));
   if(ptr == NULL)
         printf("\nOVERFLOW");
   else
   {
      printf("\nEnter the node data?");
      scanf("%d",&item);
      ptr -> data = item;
      if(head == NULL)
      {
```

```
            head = ptr;
            ptr -> next = head;
        }
        else
        {
            temp = head;
            while(temp->next != head)
                temp = temp->next;
            ptr->next = head;
            temp -> next = ptr;
            head = ptr;
        }
        printf("\nnode inserted\n");
    }
}
void lastinsert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW\n");
    }
    else
    {
        printf("\nEnter Data?");
        scanf("%d",&item);
        ptr->data = item;
        if(head == NULL)
        {
            head = ptr;
            ptr -> next = head;
        }
        else
        {
            temp = head;
            while(temp -> next != head)
            {
                temp = temp -> next;
            }
            temp -> next = ptr;
            ptr -> next = head;
        }

        printf("\nnode inserted\n");
    }
}
void begin_delete()
{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\nUNDERFLOW");
    }
    else if(head->next == head)
    {
```

```
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }
     else
    {   ptr = head;
        while(ptr -> next != head)
            ptr = ptr -> next;
        ptr->next = head->next;
        free(head);
        head = ptr->next;
        printf("\nnode deleted\n");
   }
}
void last_delete()
{
    struct node *ptr, *preptr;
    if(head==NULL)
    {
        printf("\nUNDERFLOW");
    }
    else if (head ->next == head)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");

    }
    else
    {
        ptr = head;
        while(ptr ->next != head)
        {
            preptr=ptr;
            ptr = ptr->next;
        }
        preptr->next = ptr -> next;
        free(ptr);
        printf("\nnode deleted\n");
   }
}
void search()
{
    struct node *ptr;
    int item,i=0,flag=1;
    ptr = head;
    if(ptr == NULL)
    {
        printf("\nEmpty List\n");
    }
    else
    {
        printf("\nEnter item which you want to
        search?\n"); scanf("%d",&item);
        if(head ->data == item)
        {
        printf("item found at location %d",i+1);
```

```
        flag=0;
        }
        else
        {
        while (ptr->next != head)
        {
           if(ptr->data == item)
           {
              printf("item found at location %d ",i+1);
              flag=0;
              break;
           }
           else
           {
              flag=1;
           }
           i++;
           ptr = ptr -> next;
        }
        }
        if(flag != 0)
        {
           printf("Item not found\n");
        }
     }
  }
  void display()
  {
     struct node *ptr;
     ptr=head;
     if(head == NULL)
     {
        printf("\nnothing to print");
     }
     else
     {
        printf("\n printing values ... \n");
         while(ptr -> next != head)
        {
             printf("%d\n", ptr -> data);
           ptr = ptr -> next;
        }
        printf("%d\n", ptr -> data);
     }

  }
```

Abstract Data Type (ADT)- ADT is a mathematical model that logically represents a datatype. ADT only tells us the essential features without including background details. It specifies set of data and set of operation that can be performed on the data. Example: List adt, Stack adt. Queue adt

Array Implementation of linked list

```
#include<stdio.h>
#include<stdlib.h>
```

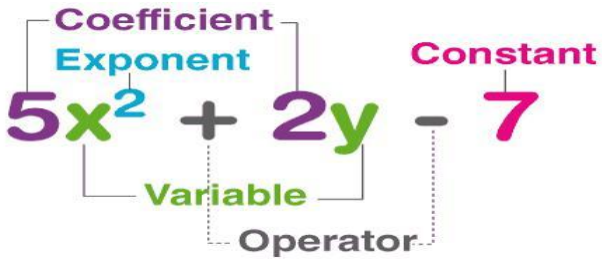
```
int INFO[20], LINK[20], START, AVAIL;
void INSLOC(int, int);
int FINDA(int);
void main()
{
int PTR, ITEM, i=0,LOC;
printf ("value of info\n ");
for(i=0;i<20;i++)
scanf("%d",&INFO[i]);
printf ("value of link\n ");
for(i=0;i<20;i++)
scanf("%d",&LINK[i]);
for(i=0;i<20;i++)
printf("%d    %d \n",INFO[i],LINK[i]);
START=11;
AVAIL=15;
PTR =START;
printf("LIST: \n");
while (PTR!=-1)
{
printf("\n%d \t", INFO [PTR]);
PTR=LINK [PTR];
}
printf("\n\nEnter the ITEM to be inserted: ");
scanf ("%d", &ITEM);
LOC=FINDA (ITEM);
INSLOC (ITEM, LOC);
PTR=START;
printf("\nModified LIST: \n");
while(PTR!=-1)
{
printf("\n%d\t", INFO[PTR]);
PTR = LINK[PTR];
}
}

void INSLOC (int T, int L)
{
int NEW;
if(AVAIL== -1)
{
printf("\nOVERFLOW");
exit(1);
}
NEW =AVAIL;
AVAIL=LINK [AVAIL];
INFO [NEW]=T;
if (L==-1)
{
LINK [NEW] =START;
 START=NEW;
}
else
{
LINK [NEW]=LINK [L];
 LINK [L]=NEW;
}}
```

```
int FINDA (int T)
{
int L, SAVE, P;
if (START==-1)
{
L=-1;
return (L);
}
if(T<INFO[START])
{
L=-1;
return (L);
}
SAVE=START;
P=LINK [START];
while(P!=-1)
{
if (T<INFO[P])
{
L=SAVE;
return (L);
}
SAVE=P;

P=LINK [P];
}}
```

Polynomial



A polynomial is a mathematical expression that contains more than two algebraic terms. It is a sum of several terms that contain different powers of the same variable. p(x)is the representation of a polynomial. A polynomial expression representation consists of at least one variable, and typically includes constants and positive constants as well. It is represented as $a_1x^n + a_2x^{n-1} + a_3x^{n-2} + \ldots\ldots\ldots\ldots + a_nx^0$,
where x is the variable, $(a_1, a_2, a_3, \ldots\ldots\ldots\ldots\ldots, a_n)$are the coefficients and n is the degree of the polynomial. The coefficients must be a real number.
It is necessary for the polynomial expression that each expression consists of two parts:

1. Coefficient part
2. Exponent part

Example:
$p(x) = 21x^3 + 3x^2 + 14x^1 + 21x^0$ .Here, 21, 3, 14, and 21 are the coefficients, and 3,2,1 and 0 are the exponential values x.

Types of polynomials

Monomial
It has only one term, like 2x, 23xy.
Binomial
It has two terms, or we can say that it is the sum of two monomials; for example, 2x+3.
Trinomial
It consists of three terms of monomials; for example, 2x+3y+7z.

Representation of polynomial

There are various ways to represent the polynomials, two of which are given below.

- Using array
- Using a linked list
-

Representation of polynomial equation using array
The operations such as addition, subtraction, multiplication, differentiation and so forth can be performed on the polynomials represented as arrays.
Example:
Consider a polynomial with two variables: $2x^2+5xy+y^2$. The array representation of the polynomial is give below:

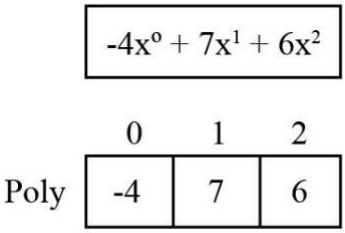
2	2	0	5	1	1	1	0	2

Index 0 stores the coefficient of the first term, index 1 stores the exponent of the variable x and index 2 stores the exponent of the variable y in the first term. The process is repeated for the remaining terms in the polynomial. It can also be represented as a 2-dimentional array as follows:

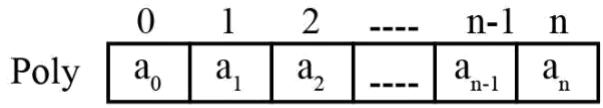
	y^0	y^1	y^2
X^0	0	0	1
x^1	0	5	0
x^2	2	0	0

Representation of polynomial with single variable -

Consider a polynomial $-4+7x+6x^2$then we can write as: $-4x^0+7x^1+6x^2$. . For the one-dimensional array, store the coefficients in the index given by the coefficient of the polynomial.



int poly [3];
array representation of polynomial
In the array representation, the array first the first element stores the coefficient of the term with the lowest exponent and the last element contains the coefficient of the term with the highest exponent. The above diagram shows the array representation of polynomial. A polynomial of a single variable A(X)can be written as $a_0+a_1X^1+a_2X^2+a_3X^3+.....+a_nX^n$ where $a_n\neq0$and degree of A(X) is n. Here a_0, a_1, a_2, ...a_n is the coefficient of respective terms.



The polynomial is represented using an array of size n which has n+1 terms.

Polynomial Addition using Array

Consider two different polynomials X1 = $3x^1+5x^2+7x^3$and X2= $7x^0+3x^1+4x^2$. The degree of X1is 3 and the degree of X2 is 2. The steps to add the polynomials are listed below.

- First identify the highest degree polynomial. The degree of the resultant polynomial is same as the polynomial with the highest degree.
- Store the coefficient in the index specified by the exponents of the polynomial.
- Add the coefficients stored in one array with the corresponding index positions in the other array and store the result in the same index position in the resultant array.

$X_1 = 3x^1 + 5x^2 + 7x^3$

$X_1 = 0x^\circ + 3x^1 + 5x^2 + 7x^3$

0	1	2	3
0	3	5	7

$X_2 = 7x^\circ + 3x^1 + 4x^2$

$X_2 = 7x^\circ + 3x^1 + 4x^2 + 0x^3$

0	1	2	3
7	3	4	0

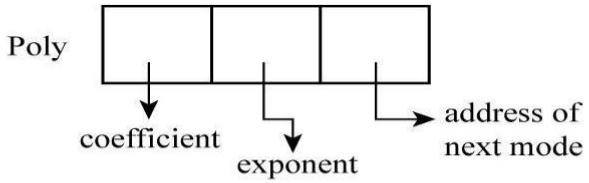
$X_3 = 7x^\circ + 6x^1 + 9x^2 + 7x^3$

0	1	2	3
7	6	9	7

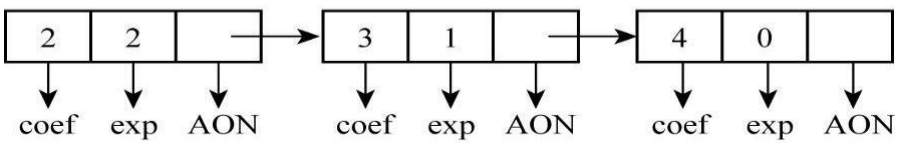
Representation of polynomial using linked list

An ordered list of non-zero terms can be thought of as a polynomial. Each non-zero term consists of three sections namely coefficient part, exponent part, and then a pointer pointing to the node containing the next term of the polynomial.

Let's take an example- If the polynomial is $2x^2+3x+$, then it is written in the form of $2x^2+3x^1+4x^0$ and represented it using a linked list. In the diagram, AON means "address of next node".

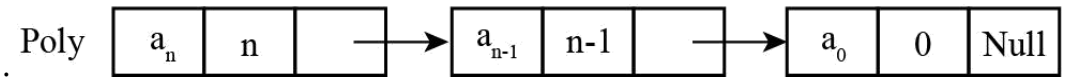


$2x^2 + 3x^2 + 4x^0$



Linked List Representation

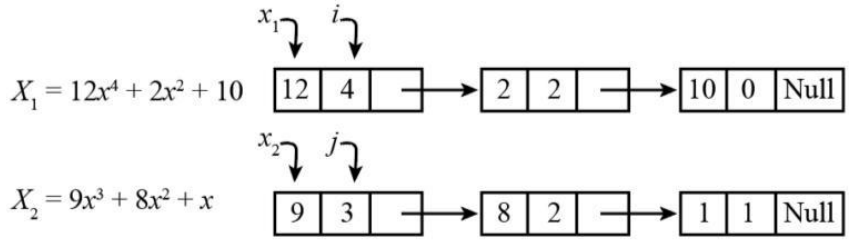
The above diagram shows the array representation of polynomial. A polynomial of a single variable A(X)can be written as $a_0+a_1X_1+a_2X_2+a_3X_3+.....+a_nX_n$ where $a_n \neq 0$and degree of A(X) is n. Here a_0, a_1, a_2, ...a_n is the coefficient of respective terms.



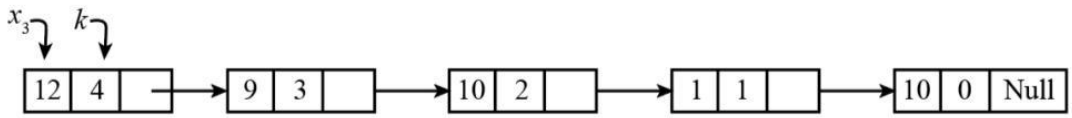
linked list representation

Addition of polynomials represented as linked lists

Consider the polynomials $12x^4+2x^2+10$and $9x^3+8x^2+x$.



The resultant liked list :-



The addition of linked list is done by three cases-

Case 1:

If the exponent of the node pointed by j of X2 is less than the exponent of the current node pointed by *i* of X1, then copy the value of current node of X1 pointed by *i* in the new node. If the new node is the first node, make it pointed by X3 and a pointer *k.* Otherwise, add the new node next to the last node.

Case 2:

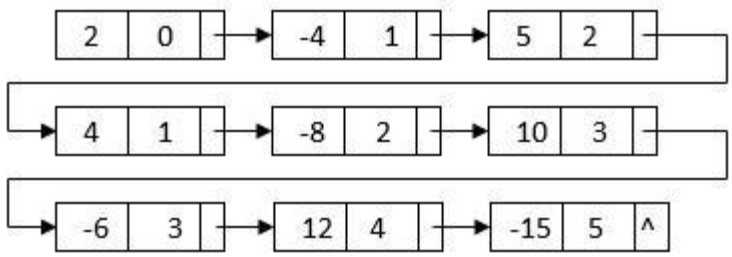
If the exponent of the node pointed by j of X2 is greater than the exponent of the current node pointed by i of X1, then copy the value of the current node X2 pointed by j in the new node. If the new node is the first node, make it pointed by X3 and a pointer k. Otherwise, add the new node next to the last node.

Case 3:

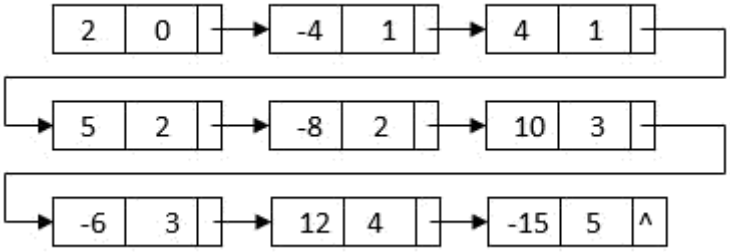
If the exponent of two terms of polynomials X1 and X2 is equal, then the coefficients are added, and the new term is stored in the resultant polynomial X3 and advance i, j and k to move to the next node.

Multiply Two Polynomials

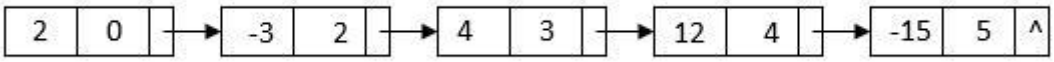
To multiply two polynomials, we can first multiply each term of one polynomial to the other polynomial. Suppose the two polynomials have and terms. This process will create a polynomial with terms. For example, after we multiply $2-4x+5x^2$ and $1+2x-3x^3$, we can get a linked list:



This linked list contains all terms we need to generate the final result. However, it is not sorted by the powers. Also, it contains duplicate nodes with like terms. To generate the final linked list, we can first **merge sort** the linked list based on each node's power:



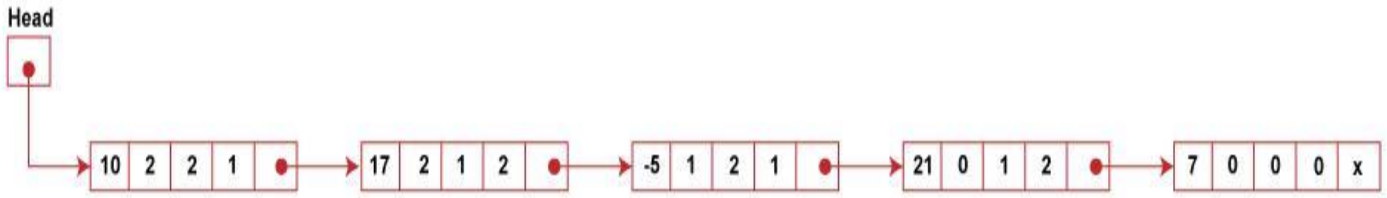
After the sorting, the like term nodes are grouped together. Then, we can merge each group of like terms and get the final multiplication result:



Polynomial of Multiple Variables

We can represent a polynomial with more than one variable, i.e., it can be two or three variables. Below is a node structure suitable for representing a polynomial with three variables X, Y, Z using a singly linked list.

Consider a polynomial P(x, y, z) = $10x^2y^2z + 17\ x^2y\ z^2 - 5\ xy^2\ z + 21y^4z^2 + 7$. On representing this polynomial using linked list are:



Terms in such a polynomial are ordered accordingly to the decreasing degree in x. Those with the same degree in x are ordered according to decreasing degree in y. Those with the same degree in x and y are ordered according to decreasing degrees in z.

Write a C program to add two polynomials using linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Structure to represent a term in a
polynomial struct node {
    int  coe;
    int exp;
    struct node* next;
};
// Function to create a new term
struct node * create (int coe, int exp)
 {
    struct node * new;
    new = (struct node *)malloc(sizeof(struct node *));
    new->coe = coe;
    new->exp = exp;
```

```
    new->next = NULL;
    return new;
}

// Function to add a term to a polynomial void
add(struct node **poly1, int coe, int exp)
{
        struct node *new = create (coe, exp);

    if (*poly1 == NULL)
    {
        *poly1 = new;
    } else {
            struct node * current = *poly1;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = new;
    }
}

// Function to display a polynomial
void displayPolynomial(struct node *poly)
{
    while (poly != NULL) {
        printf("%dx^%d ", poly->coe, poly->exp);
        if (poly->next != NULL) {
            printf("+ ");
        }
        poly = poly->next;
    }
    printf("\n");
}

// Function to add two polynomials
struct node* addPolynomials(struct node * poly1, struct node *poly2)
    { struct node *result = NULL;

    while (poly1 != NULL && poly2 != NULL) {
        if (poly1->exp > poly2->exp) {
            add(&result, poly1->coe, poly1->exp);
            poly1 = poly1->next;
        } else if (poly1->exp < poly2->exp) {
            add(&result, poly2->coe, poly2-
            >exp); poly2 = poly2->next;
        } else {
            int sum_coeff = poly1->coe + poly2->coe;
            if (sum_coeff != 0) {
                add(&result, sum_coeff, poly1->exp);
            }
            poly1 = poly1->next;
            poly2 = poly2->next;
        }
    }

    // Append any remaining terms from poly1 and
    poly2 while (poly1 != NULL) {
```

```
        add(&result, poly1->coe, poly1->exp);
        poly1 = poly1->next;
    }

    while (poly2 != NULL) {
        add(&result, poly2->coe, poly2->exp);
        poly2 = poly2->next;
    }

    return result;
}

int main() {
    struct node *poly1 = NULL;
    struct node *poly2 = NULL;
    struct node *result = NULL;

    // Adding terms to the first
    polynomial add(&poly1, 5, 2);
    add(&poly1, -3, 1);
    add(&poly1, 2, 0);

    // Adding terms to the second
    polynomial add(&poly2, 4, 3);
    add(&poly2, -1, 2);
    add(&poly2, 7, 0);

    printf("First polynomial: ");
    displayPolynomial(poly1);

    printf("Second polynomial: ");
    displayPolynomial(poly2);

    result = addPolynomials(poly1, poly2);

    printf("Resultant polynomial: ");
    displayPolynomial(result);

    free(poly1);
    free(poly2);
    free(result);

    return 0;
}
```